

Chapter 5

Unsupervised Machine Learning



Generation AI

Nurturing Future Innovators

OTHER LEARNING PARADIGMS

«Սույն նյութի, ներառյալ դրա ամբողջ բովանդակության, ձևավորման, տեքստերի, պատկերների և ցանկացած այլ տարրի հեղինակային իրավունքը պատկանում է Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամին:

Սույն նյութը չի կարող ամբողջությամբ կամ մասնակիորեն վերարտադրվել, տարածվել, փոխանցվել, վերախմբագրվել, թարգմանվել կամ որևէ այլ ձևով օգտագործվել առանց Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամի նախնական գրավոր համաձայնության՝ անկախ օգտագործման նպատակից:

Սույն նյութի մեջ արտահայտված տեսակետները, վերլուծություններն ու մեկնաբանությունները հանդիսանում են հեղինակների մտավոր սեփականությունը:

© 2025, Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամ: Բոլոր իրավունքները պաշտպանված են:»

Session 1: Clustering

Discovering Patterns in Unlabeled Data

Հեղինակային իրավունքը պատկանում է «Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամ»-ին և ենթակա չէ ազատ տարածման:



The Objective of the Session

- Differentiate between supervised and unsupervised machine learning
- Discuss different applications of unsupervised machine learning
- Implement the process of clustering on sample data manually



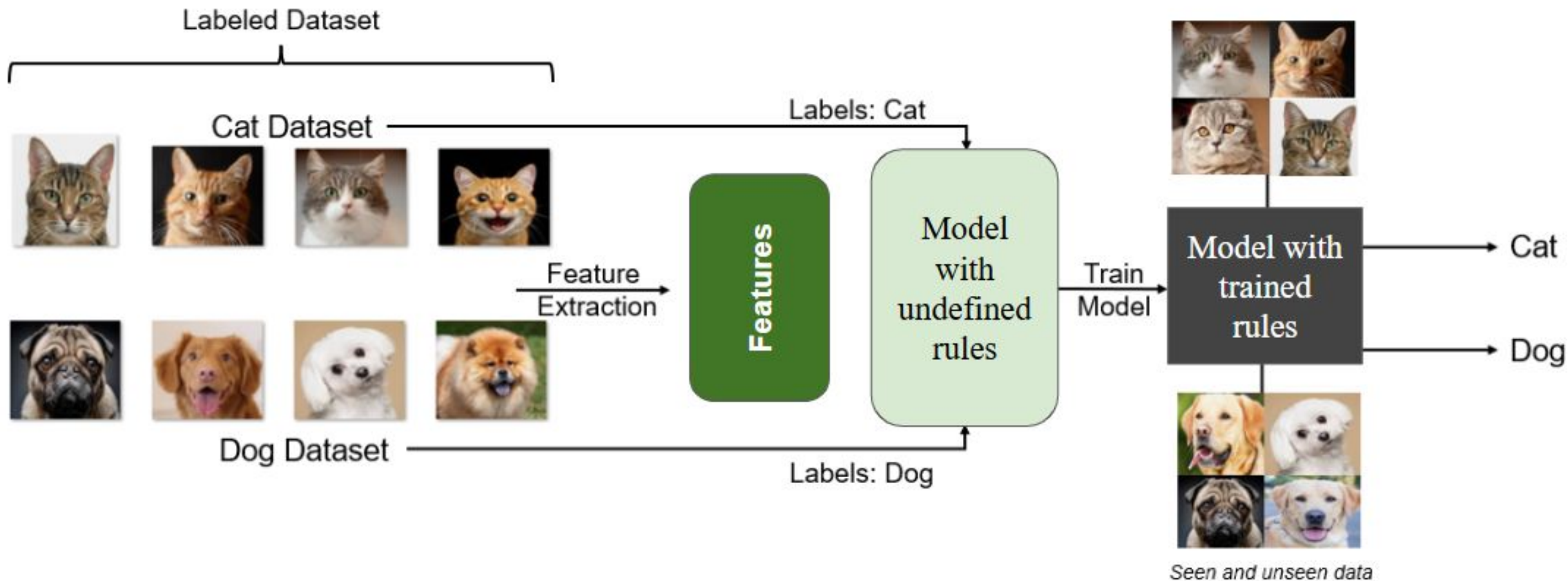
Unsupervised VS Supervised Learning

So far, we have explored several machine learning algorithms—such as

- Decision Trees,
- KNN,
- Regression,
- Neural Networks
- CNNs

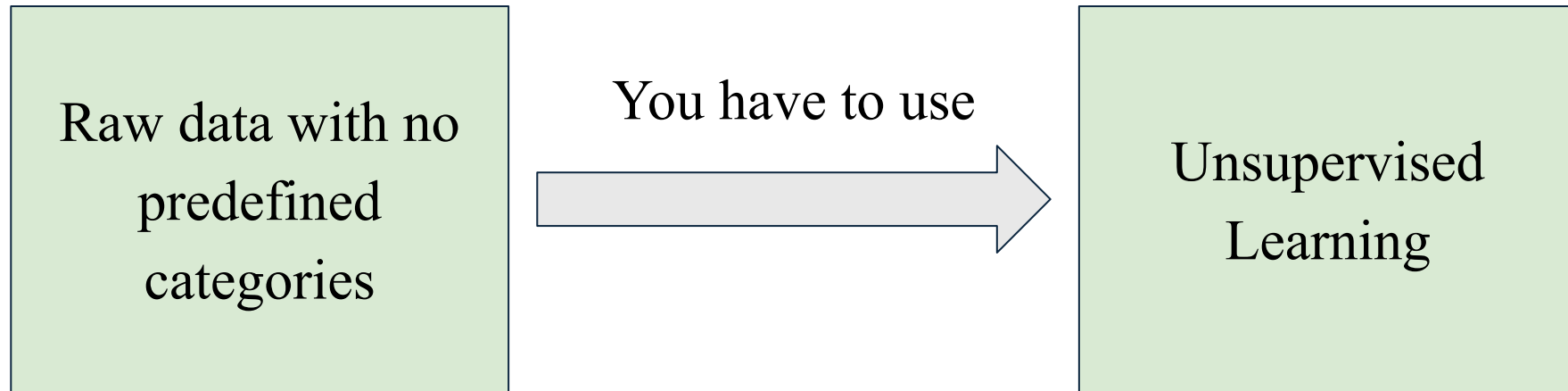
where training is guided using labeled data. *The labels of each data sample was provided as an input to the model.* These are known as supervised learning algorithms, as the learning process relies on the supervision provided by the labels to tune the model's parameters.

Unsupervised VS Supervised Learning



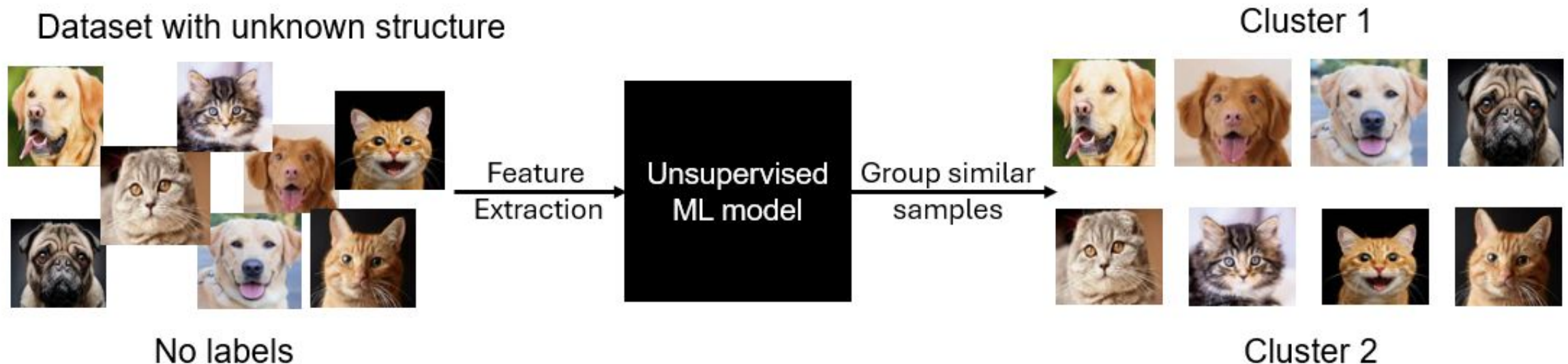
Unsupervised VS Supervised Learning

But what happens when labels are not available?



Learning Without Labels

Unsupervised learning mirrors the way children naturally differentiate and group items based on shared characteristics, without explicit instruction.

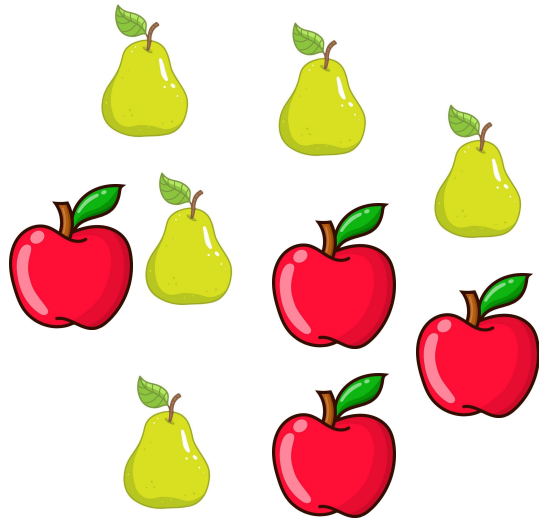


Unsupervised machine learning applications

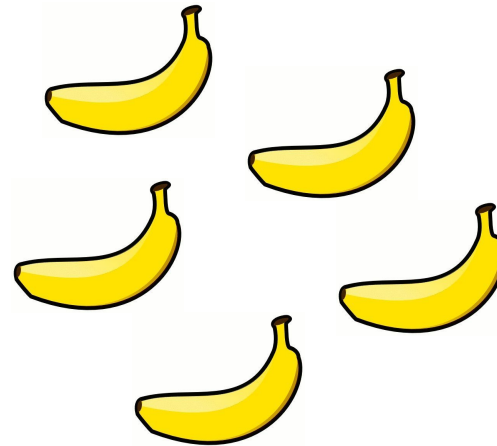
Clustering Similar Objects

In this process, we analyze the features of each fruit—such as color, size, shape, or texture—and group fruits that share similar characteristics into the same cluster.

Group 1



Group 2



This helps the computer discover patterns in the data without being told what each fruit is, just like organizing a basket of mixed fruits by "what looks alike."

Clustering Similar Objects

A movie recommendation system using clustering groups users based on their viewing habits or movie preferences is another use case of unsupervised learning.

Group 1: Action & Comedy Lovers

User 1: "I like action and comedy."

User 2: "I really enjoy fast-paced action movies and light-hearted comedies."

User 3: "Comedy and action are my go-to genres."

Group 2: Drama & Romance Lovers

User 4: "I love emotional stories and romantic plots."

User 5: "Drama and romance movies are my favorites."

User 6: "I'm drawn to heartfelt dramas and love stories."

Clustering Similar Objects

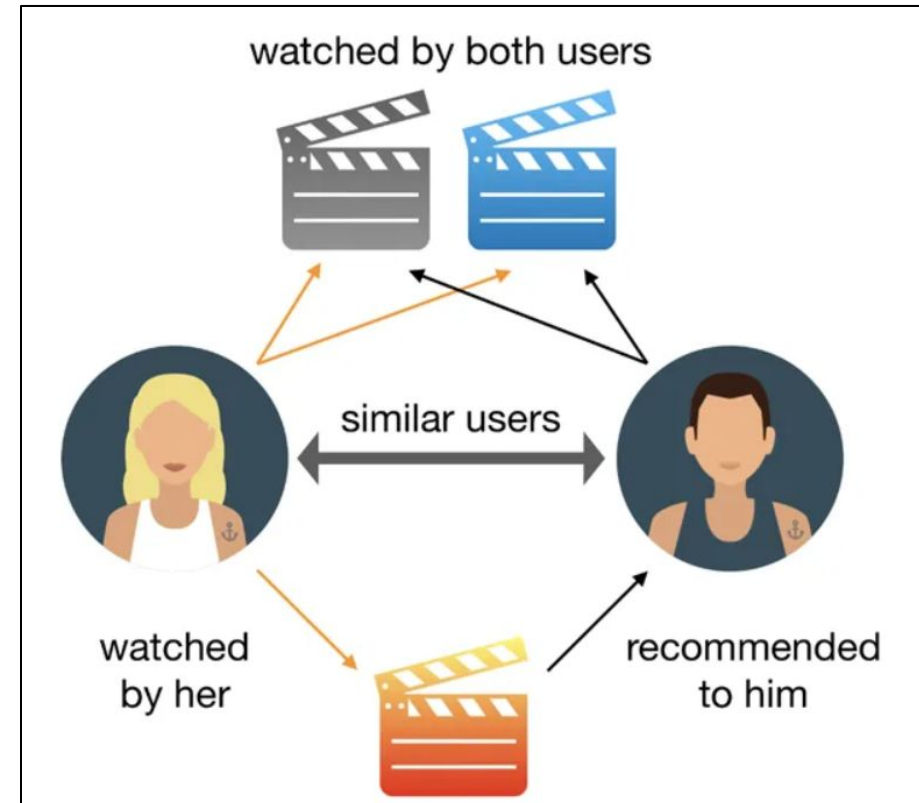
When a new movie is released or a user watches something new, the system checks their cluster and recommends movies that others in the same group have enjoyed.

Recommender systems:

Cluster users with similar preferences to recommend products or content based on history or preference.

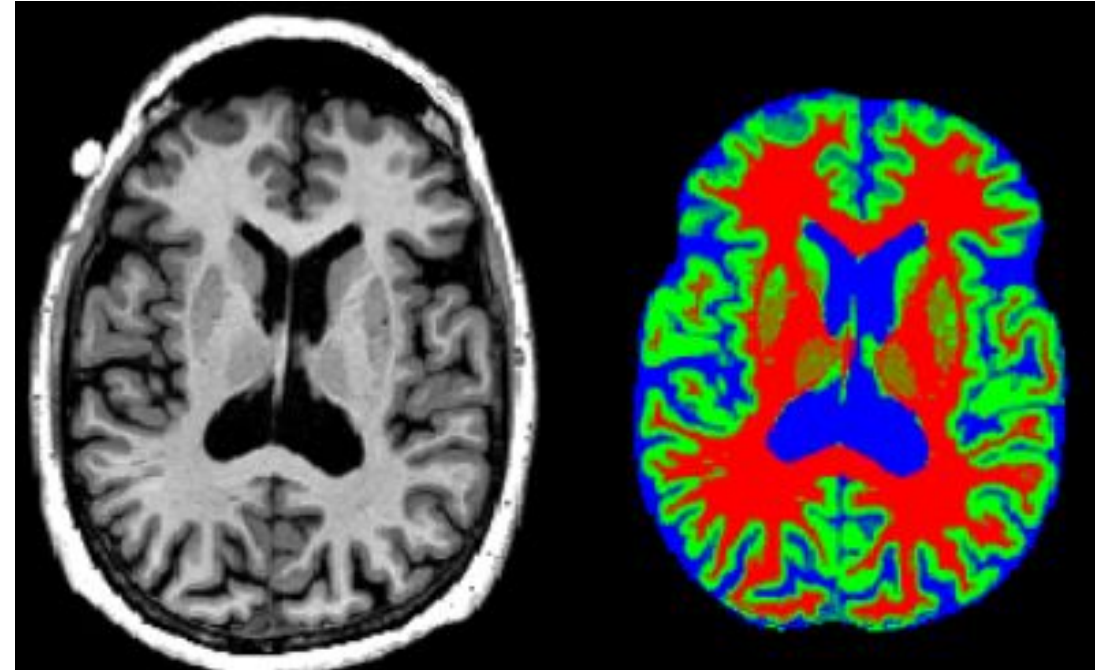
Recommends:

- Movies
- Restaurants
- Adds
- ...



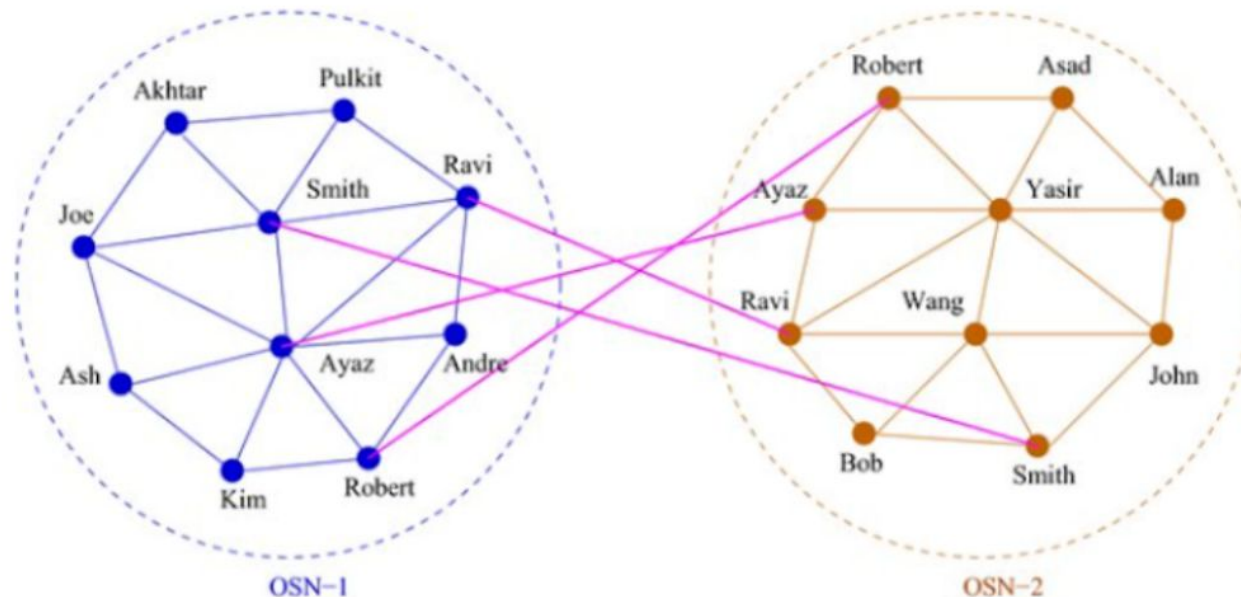
Clustering Similar Objects

Brain image segmentation in the medical field involves dividing a brain scan (like MRI) into meaningful regions based on pixel values. Each pixel in the image represents tissue with specific intensity or characteristics (e.g., gray matter, white matter, cerebrospinal fluid, or tumor). By analyzing similarities in pixel values—such as brightness or texture—clustering algorithms can group similar pixels together. This allows doctors or AI systems to automatically highlight and separate different brain structures or abnormalities, aiding diagnosis, treatment planning, and research.



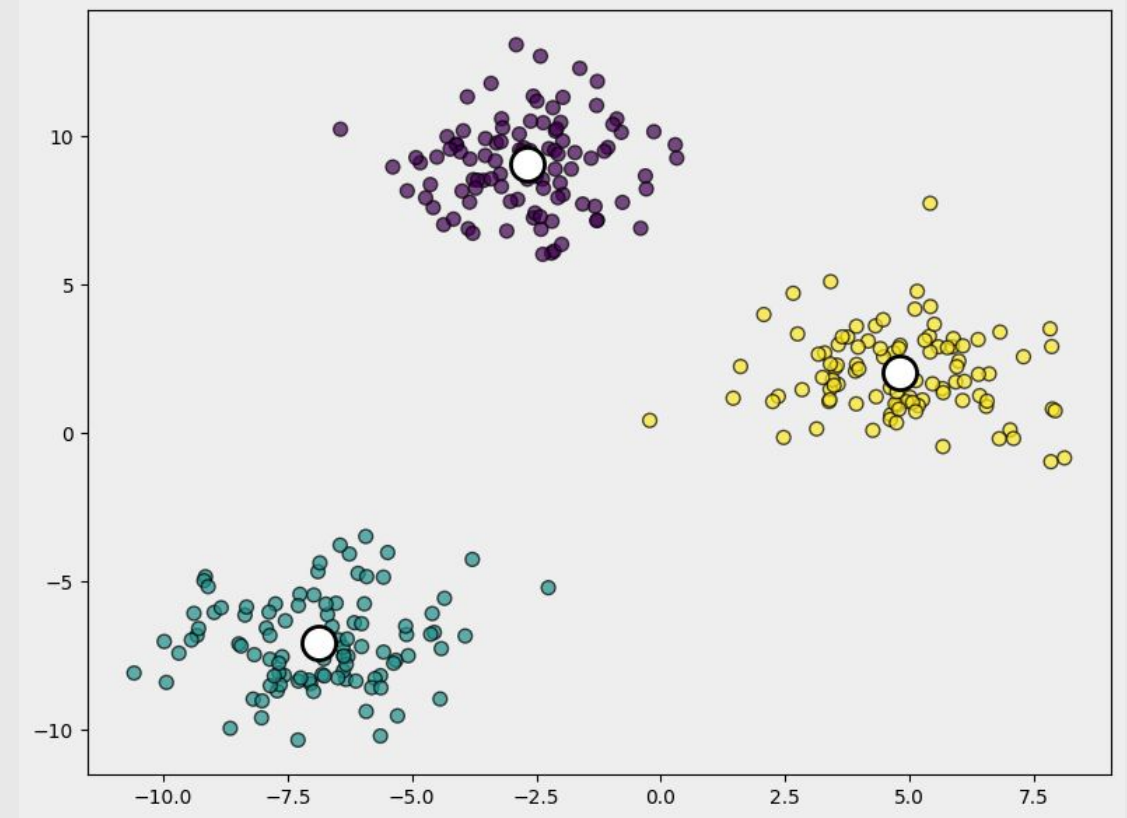
Clustering Similar Objects

Social communities are detected by analyzing patterns of interactions between individuals in a social network—like messages, likes, follows, or shared interests. Using clustering, the system identifies groups of people who interact more frequently or share similar behaviors. These clusters represent communities—such as ***friend circles***, ***interest groups***, or ***professional networks***—without needing any labels. This helps platforms recommend connections, personalize content, or understand group dynamics.



HANDOUT

In this activity, you'll explore how unsupervised learning works by performing **clustering** — a process where data is grouped based on shared features without using any labels.



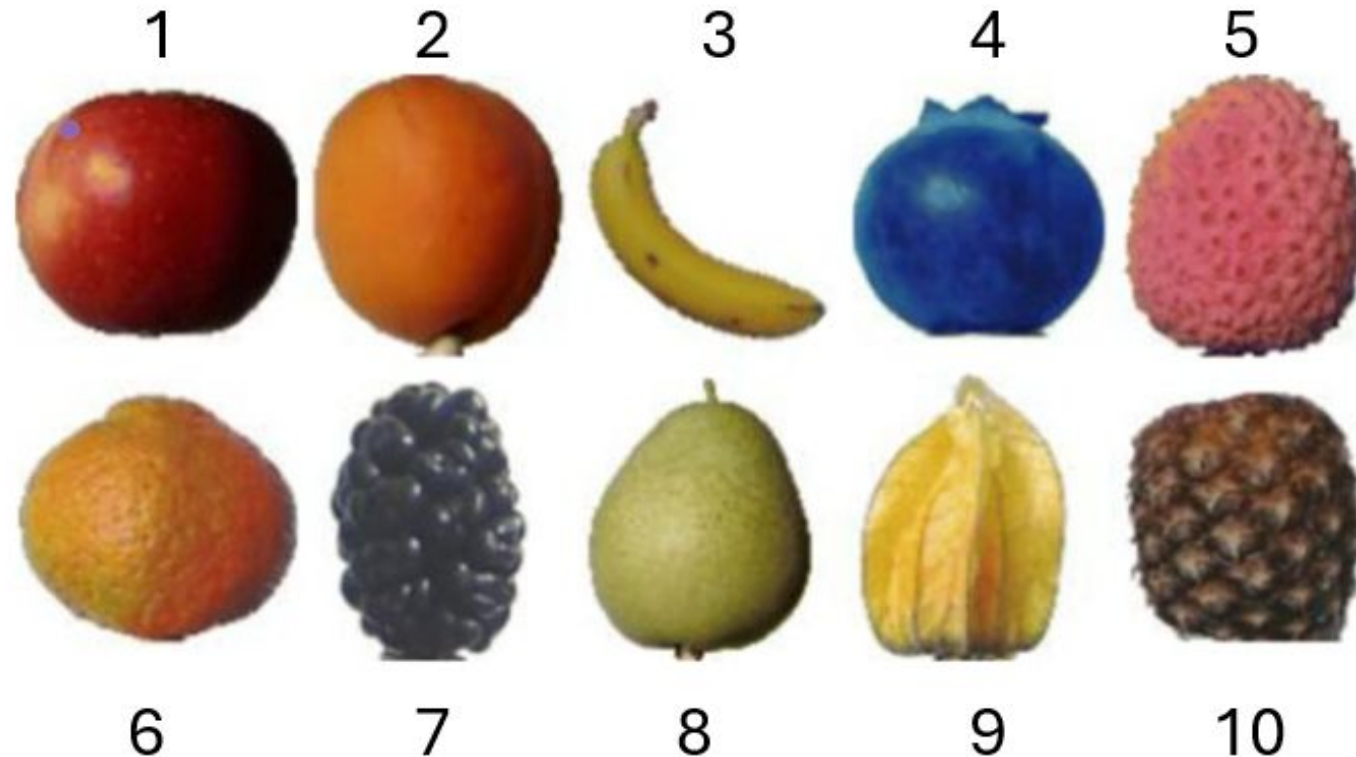
What You'll Be Doing

1. **Explore images** of animals or fruits
2. **Identify and select features** — like size, shape, or color
3. **Group similar items** into clusters based on those features
4. **Analyze your clusters** by calculating average feature values
5. **Apply your insights** to classify new, unlabeled items

Remember: There are no right or wrong answers here — different groups may choose different features and form different clusters. That's exactly what we want!

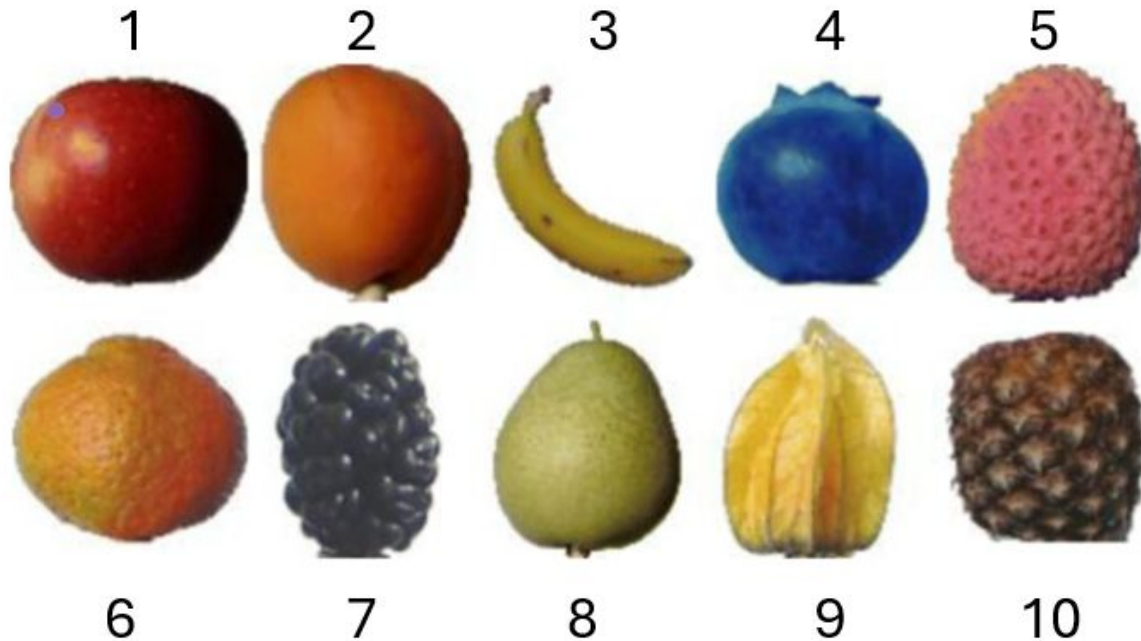
Step 1: Group & Cluster

Consider the following set of images of 10 fruits numbered from 1-10.



Step 1: Group & Cluster

As a group, observe the images and discuss *2 features* that seem most important to you.

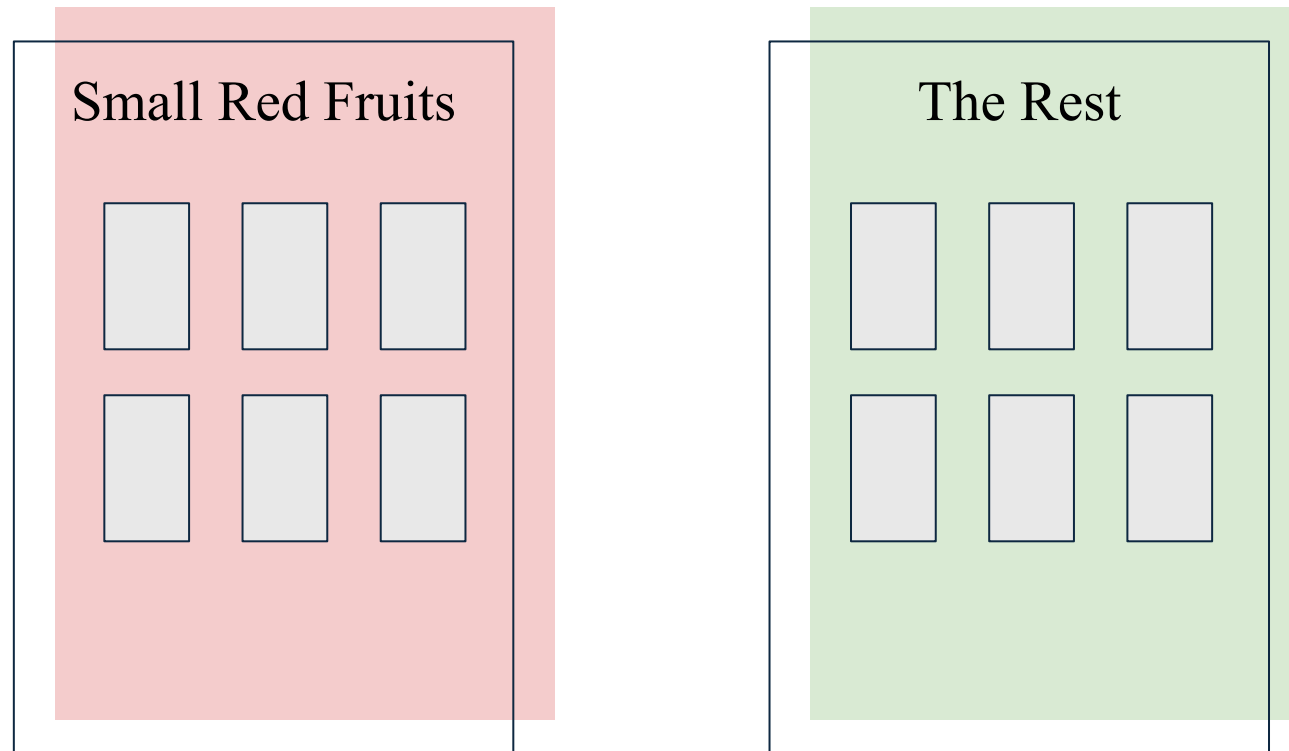


- size
- color
- shape
- texture

Step 1: Group & Cluster

Use the selected features to divide the images into two clusters (*Cluster 1* and *Cluster 2*).

Give each cluster a name or description that reflects the common features (for example, "small red fruits")



Step 2: Quantify & Average

For each selected feature, assign simple numeric values.

For Example:

Size

Small = 1

Medium = 2

Large = 3

Color

Red = 1

Yellow = 2

Green = 3

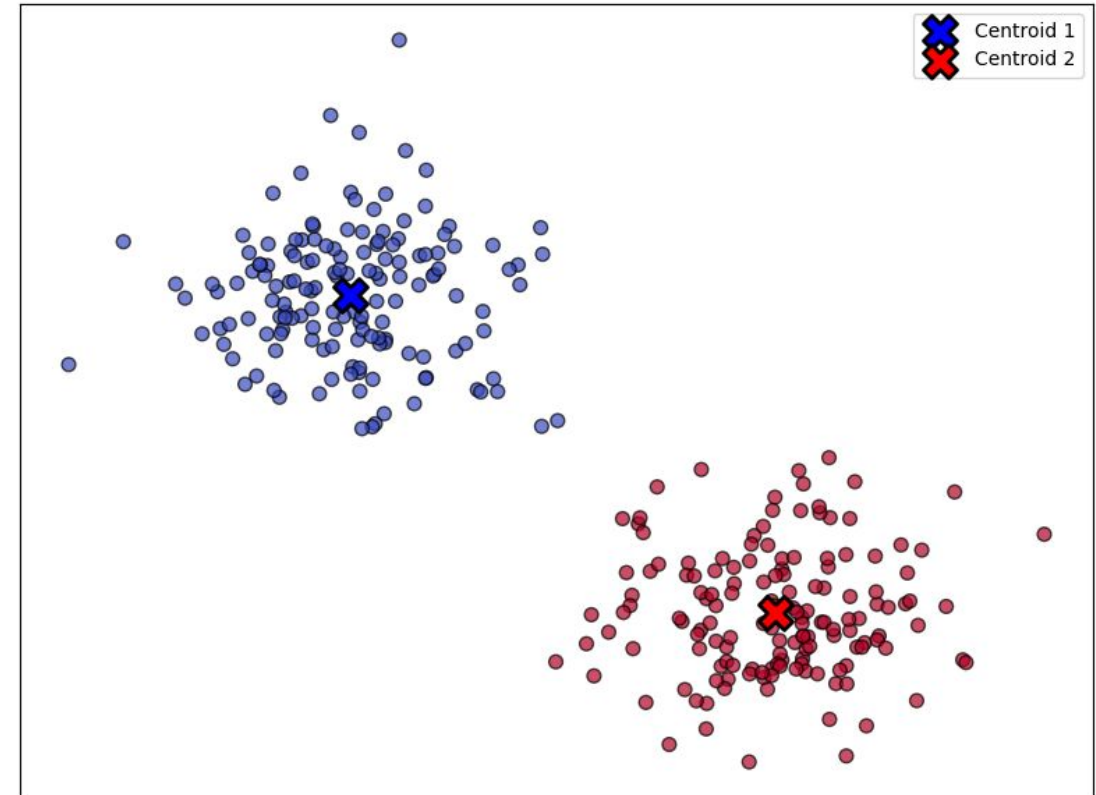
Step 2: Quantify & Average

Create a table to show the feature values of all items in each cluster.

Samples	Feature 1:	Feature 2:	Cluster #
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
Average in cluster 1			
Average in cluster 2			

Step 2: Quantify & Average

1. *Calculate the average* of each feature for both Cluster 1 and Cluster 2.
2. These averages represent the *center* or "prototype" of each cluster.
3. Plot the points (F1, F2) on an x-y plane, coloring the elements of Cluster 1 in red and the elements of Cluster 2 in blue. Mark the cluster centers with 'X' symbols, using the same color as their respective clusters.



Step 3: Classify New Samples

For the following two images

1. Estimate or *assign the feature values* for each new image.
2. Compare these feature values with your *cluster averages*.
3. Decide whether each new image belongs to *Cluster 1* or *Cluster 2*, based on which average it's closest to.



Step 4: Reflect & Share

Now, your group should *prepare a brief report* addressing the following questions:

1. What features did your group choose, and why were these features selected?
2. What did each cluster represent based on your results?
3. How did you approach classifying new items using your clusters?
4. Did other groups select different features or form different clusters? If so, how did their choices affect the outcomes?

Session 2: The K-Means Algorithm

Հեղինակային իրավունքը պատկանում է «Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամ»-ին և ենթակա չէ ազատ տարածման:



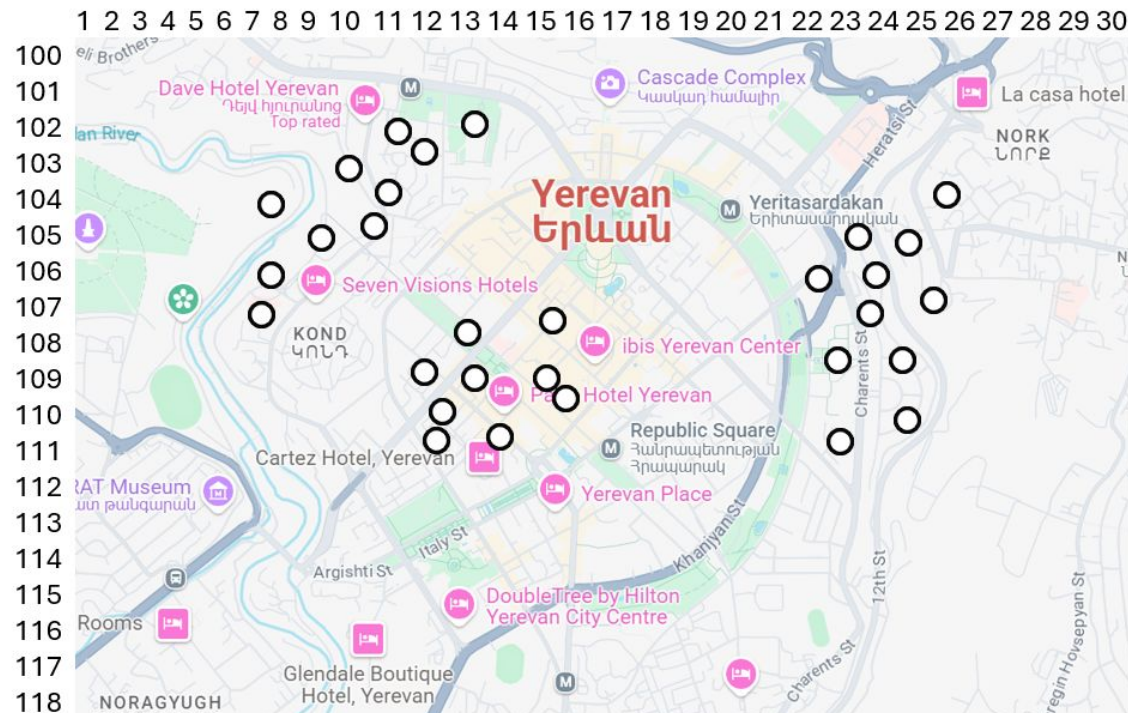
The Objective of the Session

- Identify clusters using distance.
- Formulate the k-means algorithm.
- Describe the elbow method for selecting the best number of clusters k .



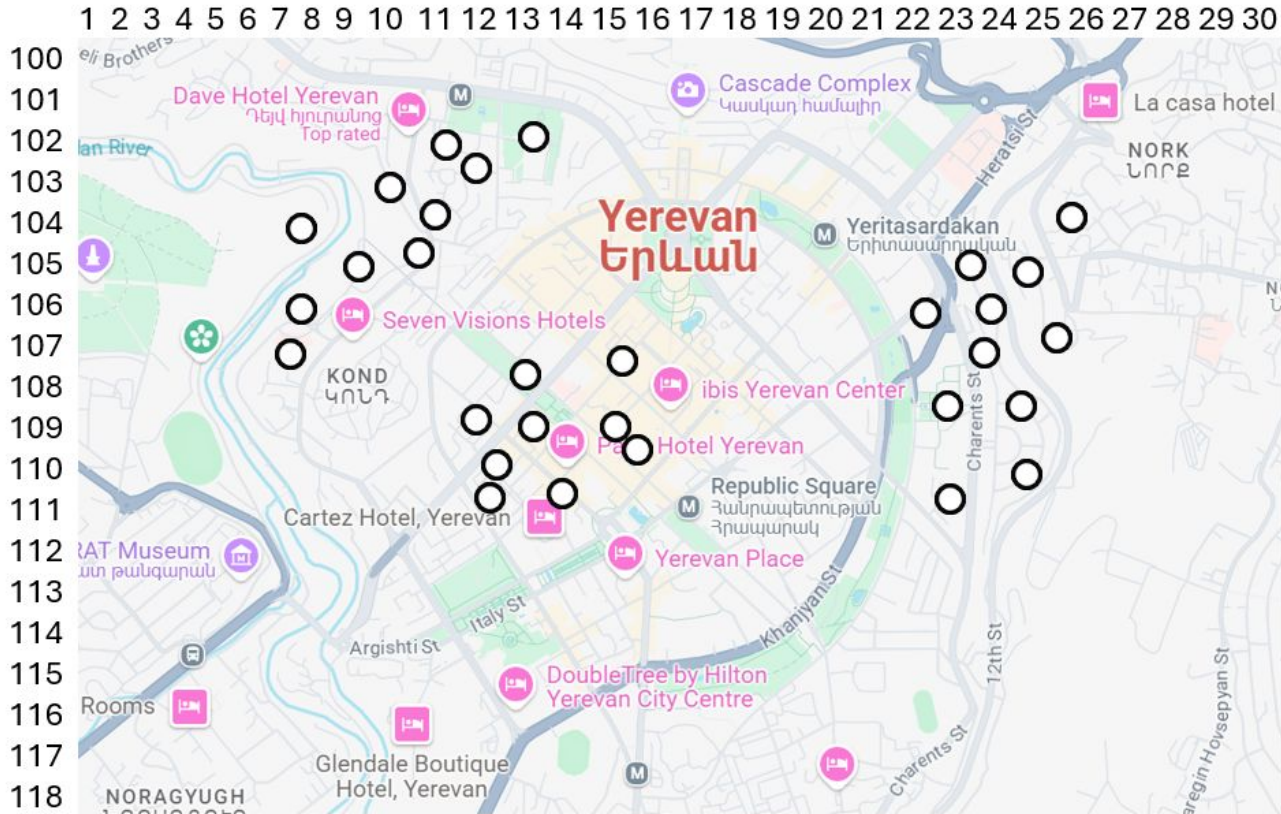
Expanding Our Pizza Delivery Business

You work for a successful pizza business in Yerevan, and your next goal is to expand by opening three new delivery centers. Your aim is to place them in locations that allow you to deliver pizzas faster by being closer to where most orders come from.



You've collected past delivery data, and the black circles on the map of Yerevan show locations where orders were delivered over the last year.

Question N1: Where should you place the 3 new delivery centers?



Mark ideal center locations (e.g., with red stars or dots) or list their ***x-y coordinates*** (e.g., [116, 21]).

Justify your choices – Did you pick spots to reduce travel time or based on delivery clusters?

Question N1: How do you think computers can do this on their own?

Now that you've manually chosen the best locations for new pizza centers based on past delivery data, imagine a computer was asked to solve the same problem.

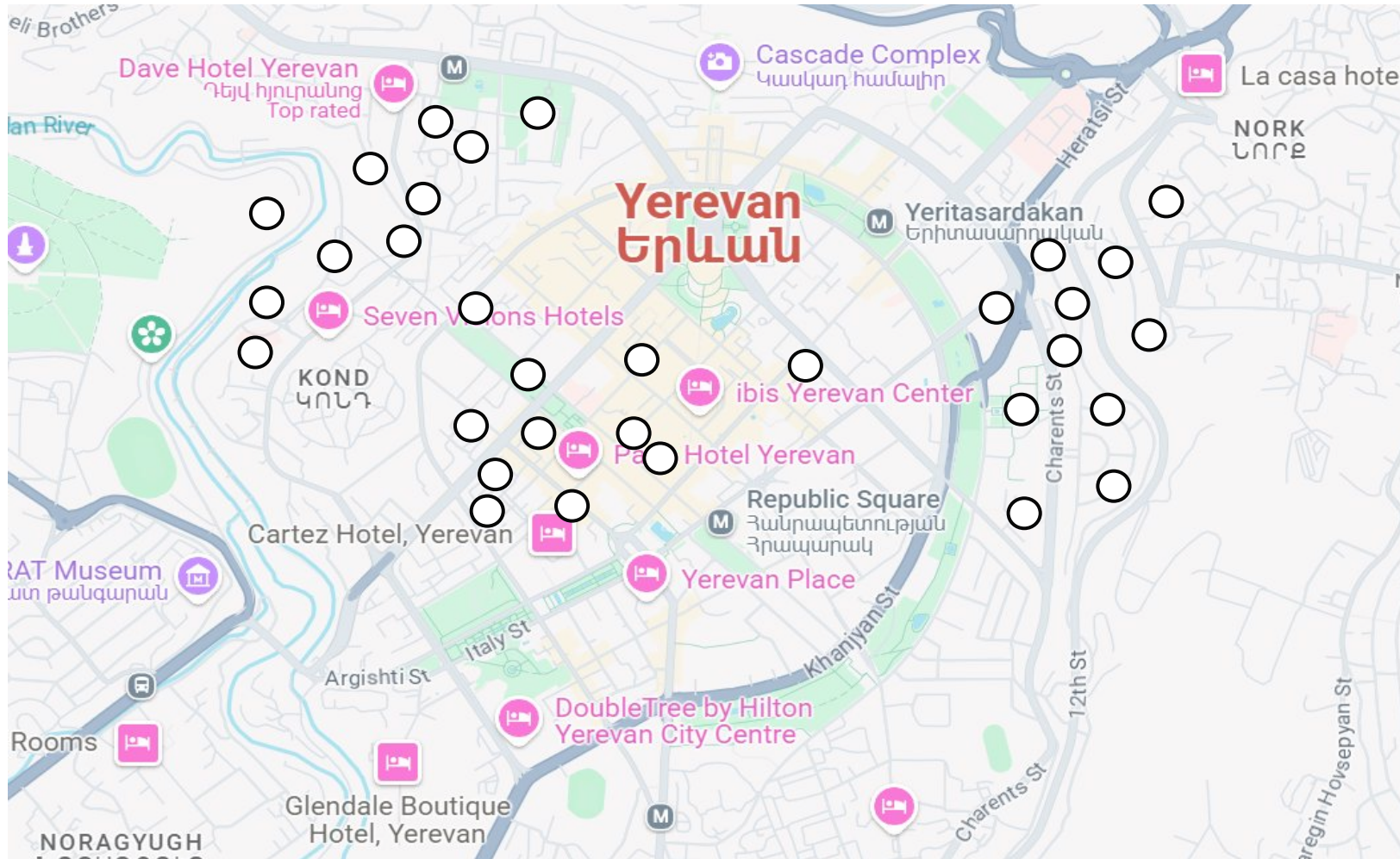
- What steps do you think the computer would take?
- What kind of data would it need?
- How might it "decide" where the centers should be placed?
- Would it follow a rule, repeat a process, or learn from patterns?

K - Means Algorithm

Now that you've manually chosen the best locations for new pizza centers based on past delivery data, imagine a computer was asked to solve the same problem.

- What steps do you think the computer would take?
- What kind of data would it need?
- How might it "decide" where the centers should be placed?
- Would it follow a rule, repeat a process, or learn from patterns?

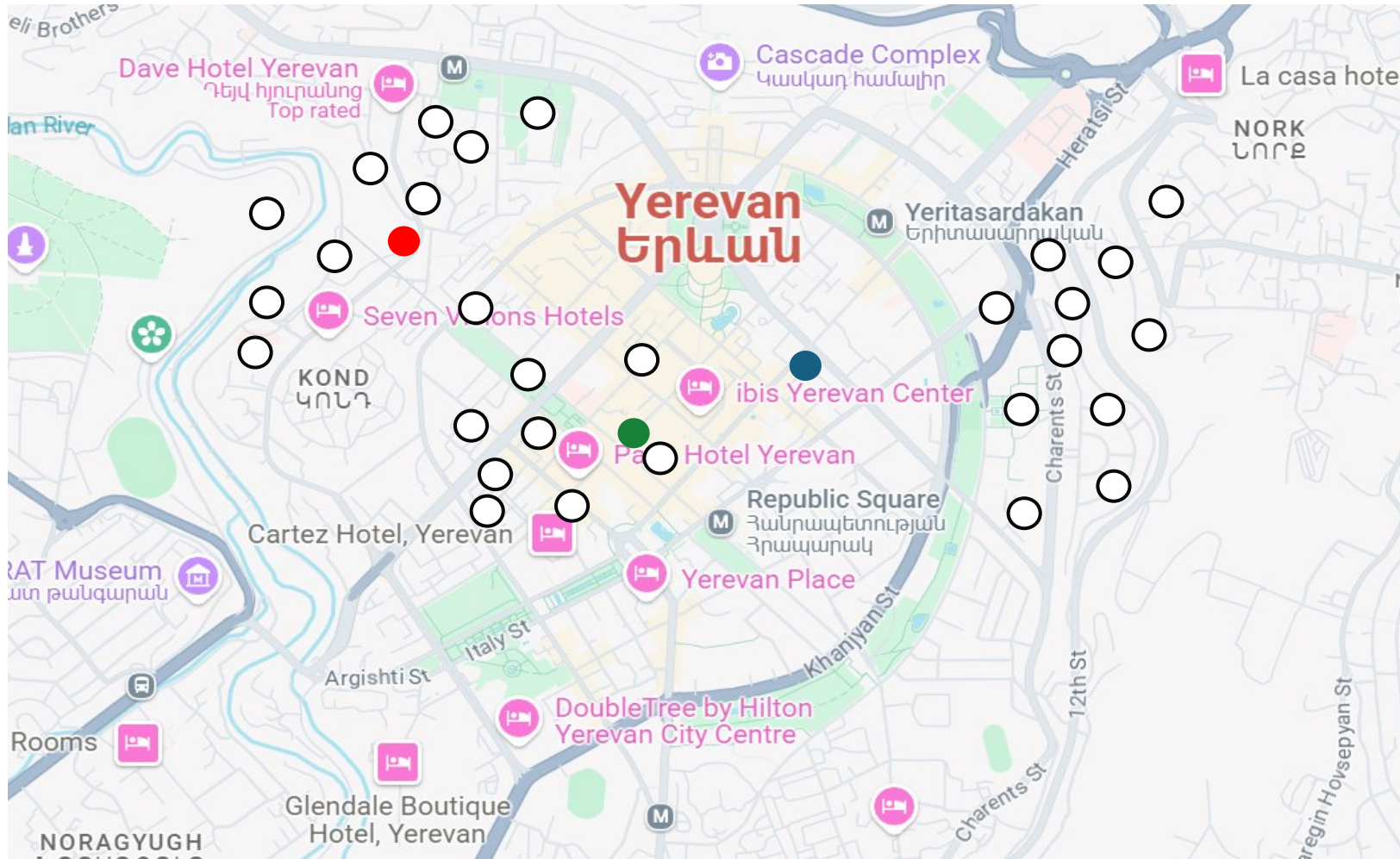
Step 1: Choose the number of clusters K .



Decide how many clusters you want to identify in the data. This is a user-defined number.

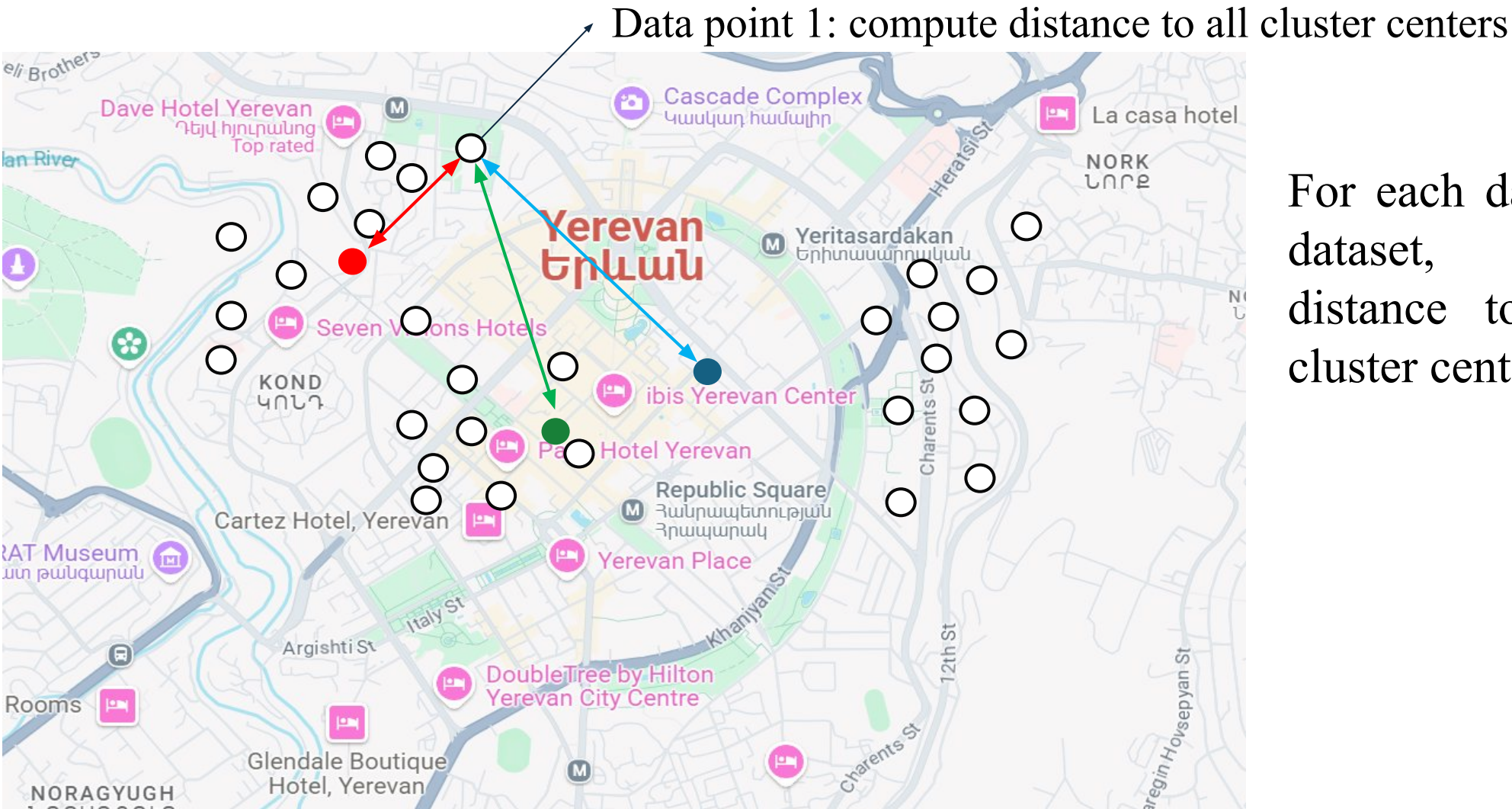
After plotting, we can presume there are three clusters. *We set $k = 3$.*

Step 2: Initialize cluster centers.



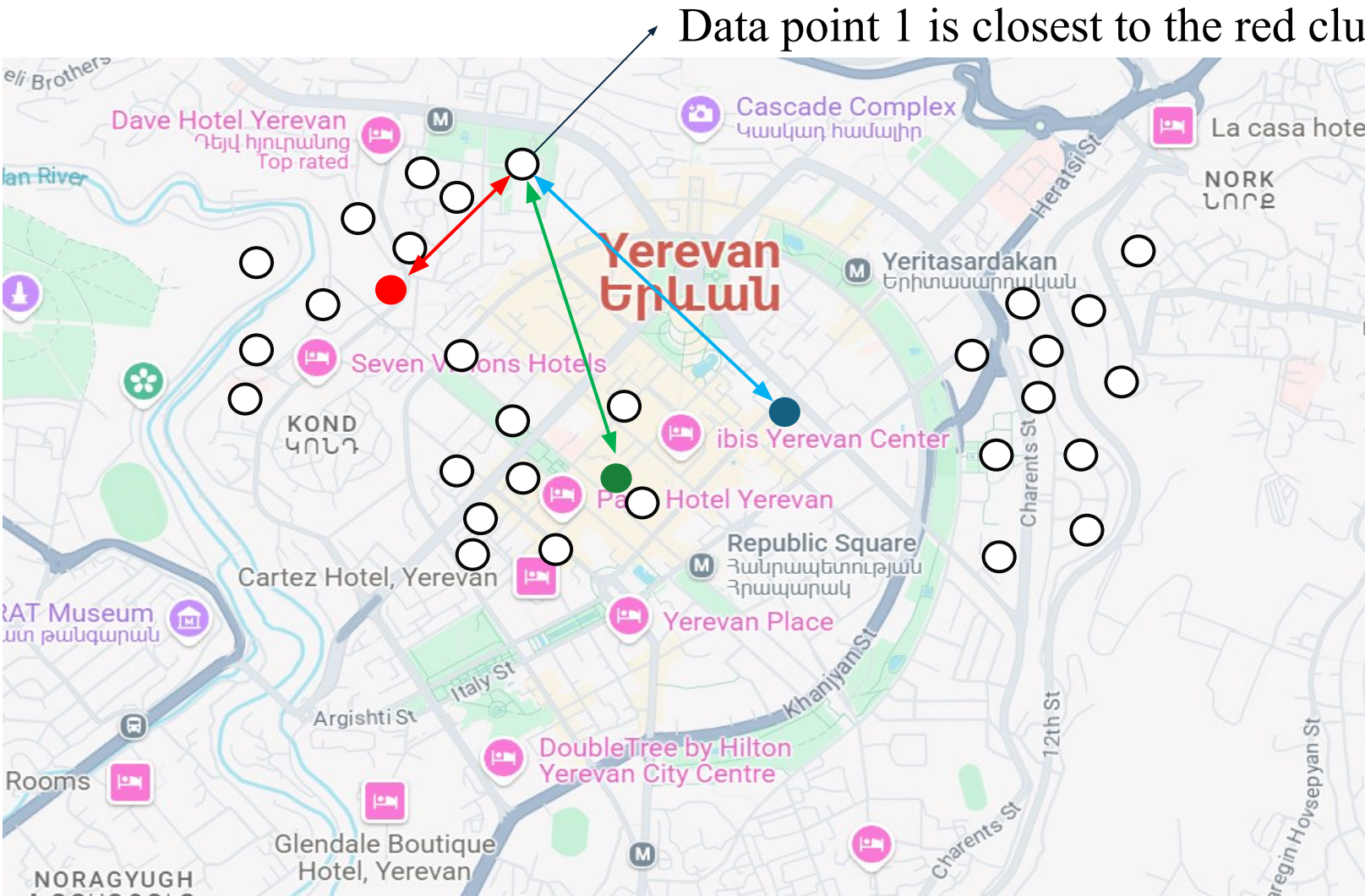
Randomly select k distinct data points from the dataset to serve as the initial cluster centers (also called centroids).

Step 3: Compute distances.



For each data point in the dataset, compute its distance to each of the cluster centers.

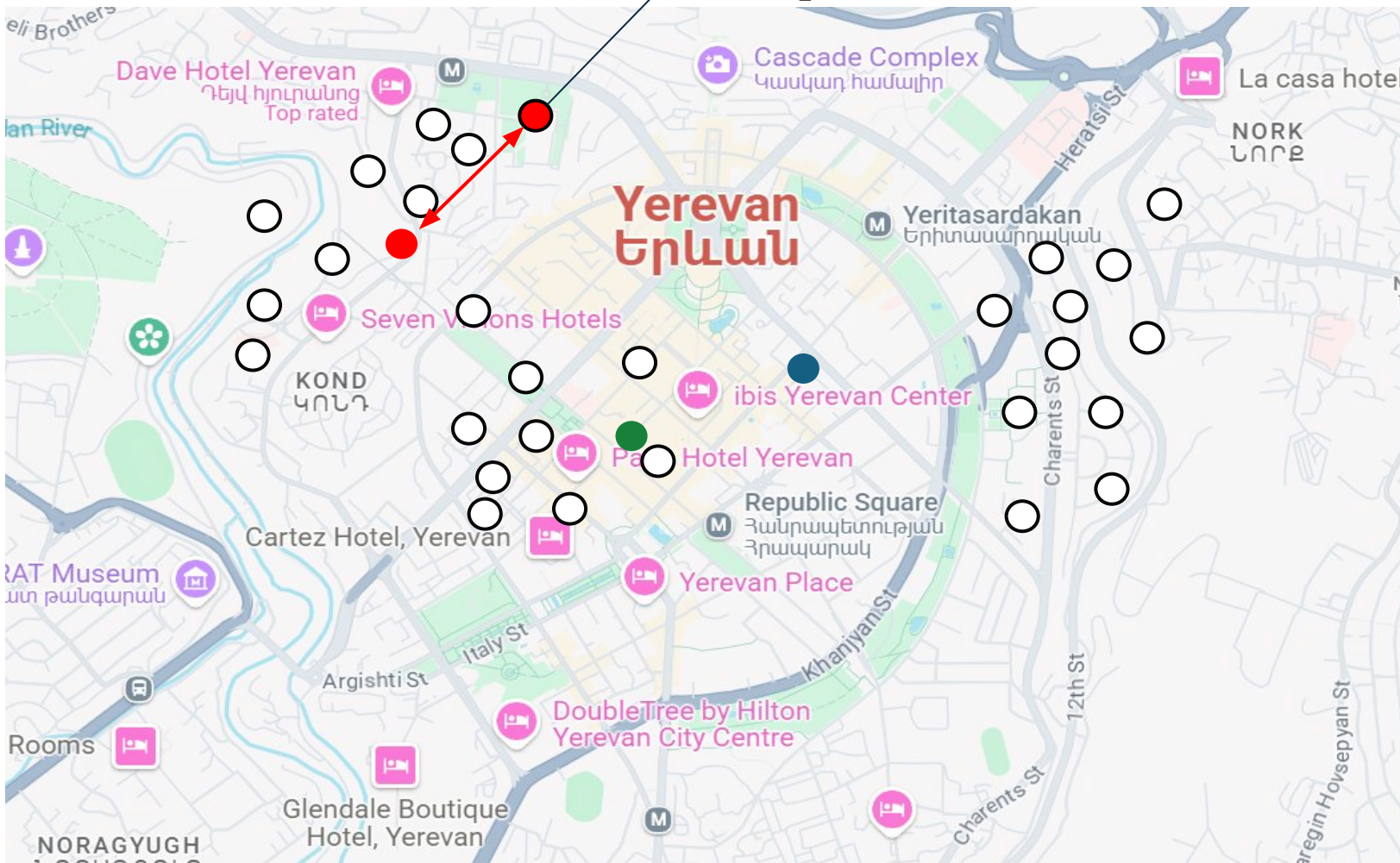
Step 4: Compute minimum distance and color the point.



Assign each data point to the cluster whose center is closest.

Step 4: Assign point 1 to a cluster.

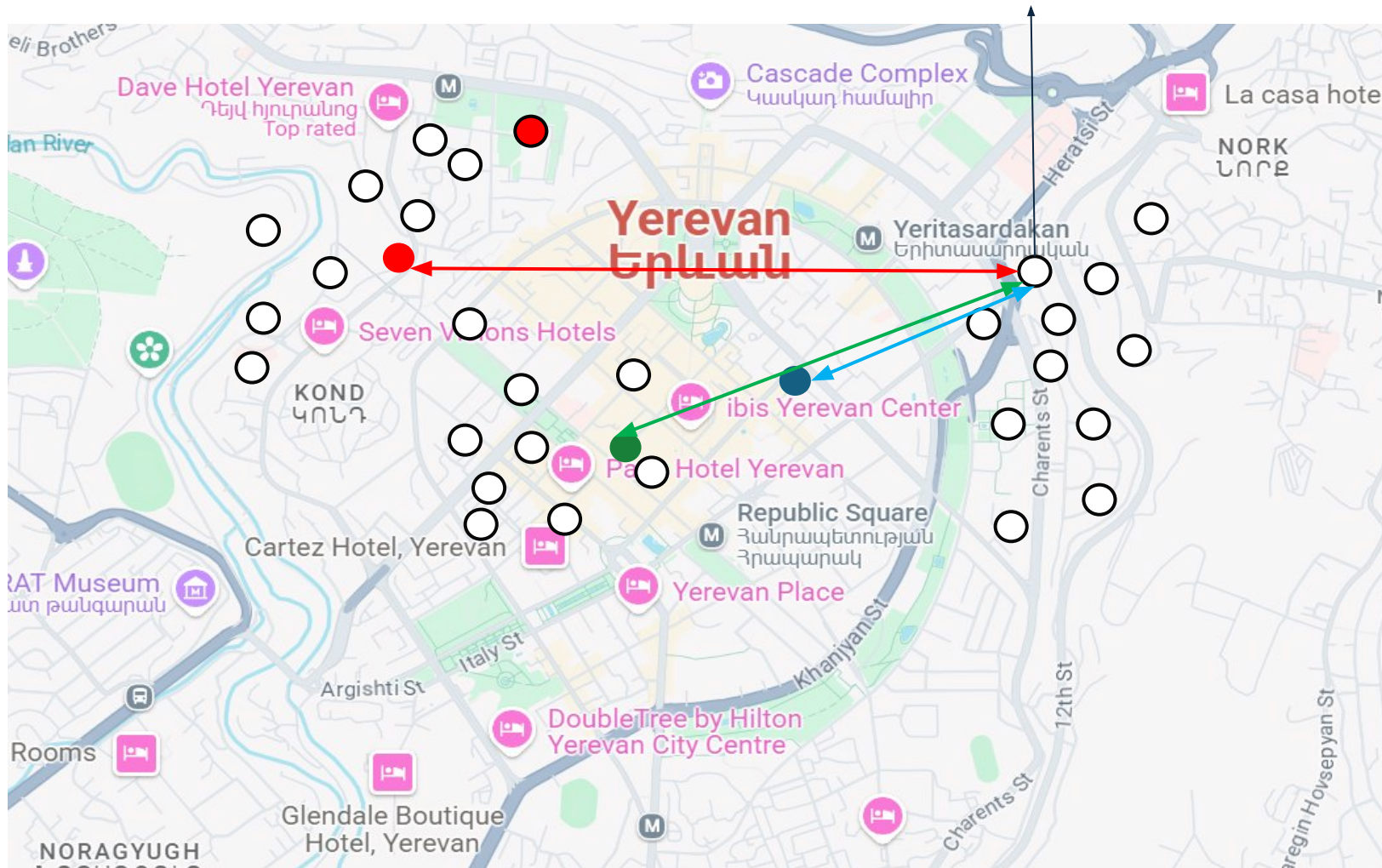
Data point 1 is closest to the red cluster center $\Rightarrow$ It is in the red cluster



Take the next point.

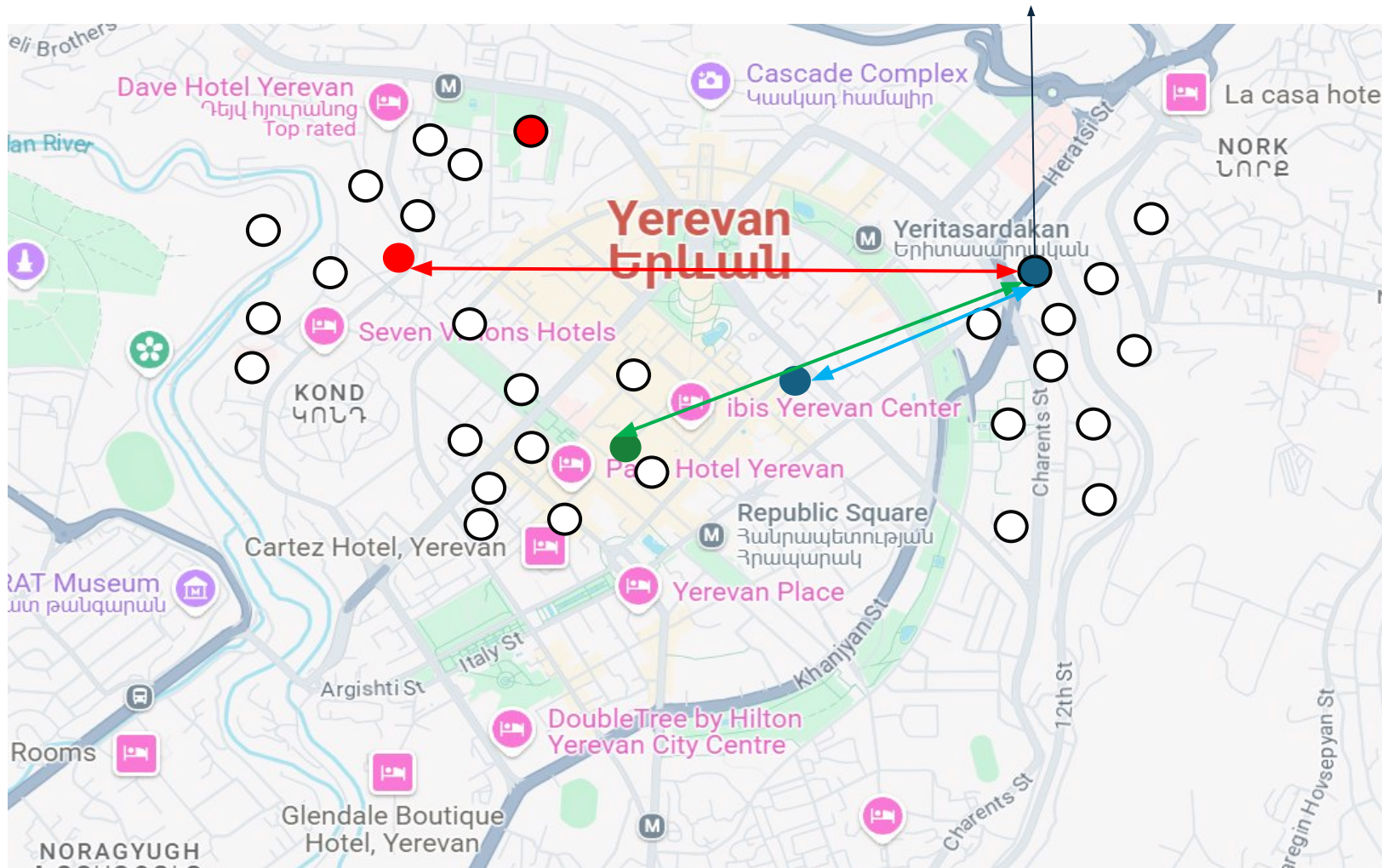
Step 4: Compute minimum distance and color the point.

Data point 2: compute distance to all cluster centers

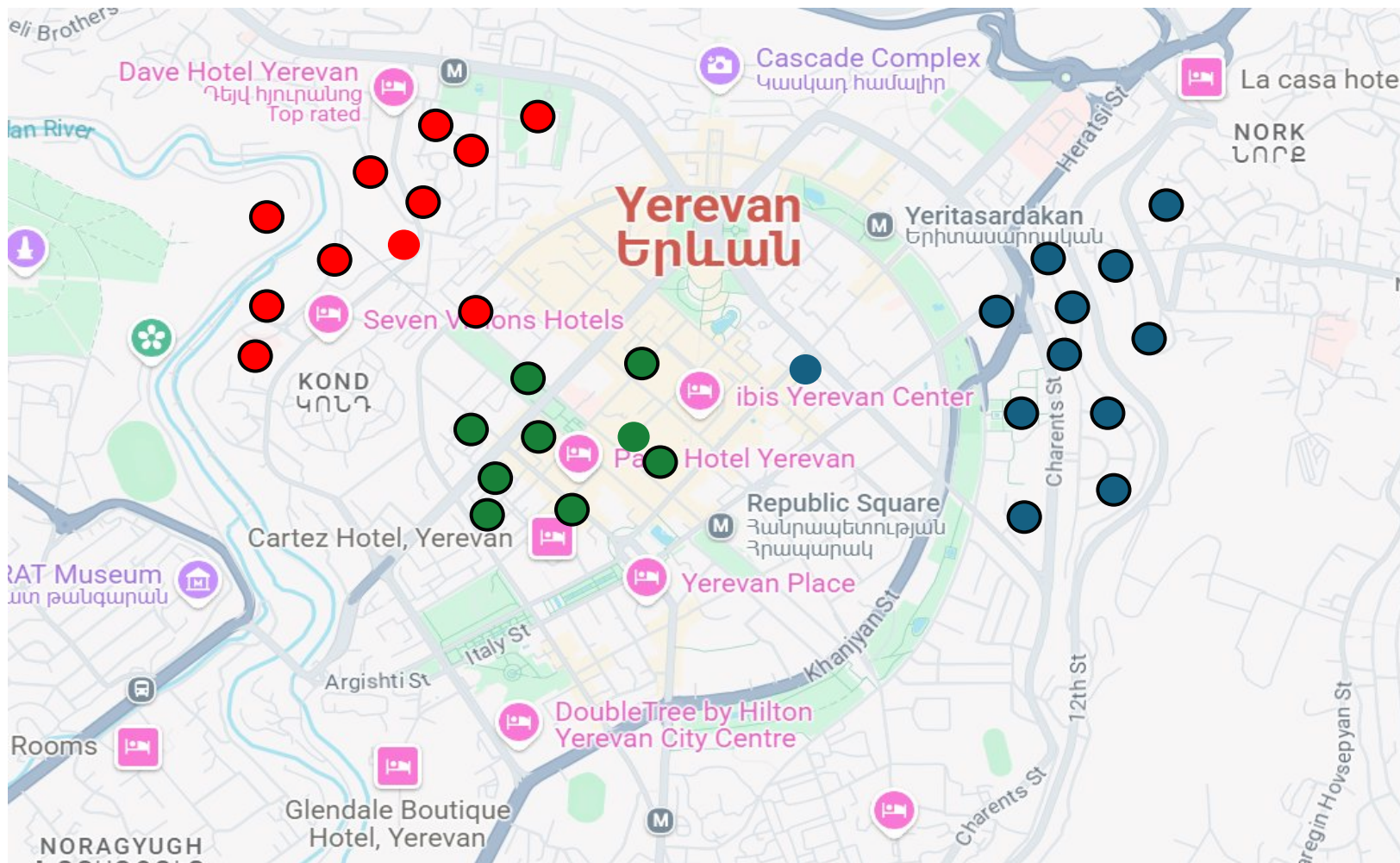


Step 4: Compute minimum distance and color the point.

Data point 2 is closest to the blue cluster center $\Rightarrow$ It is in the blue cluster

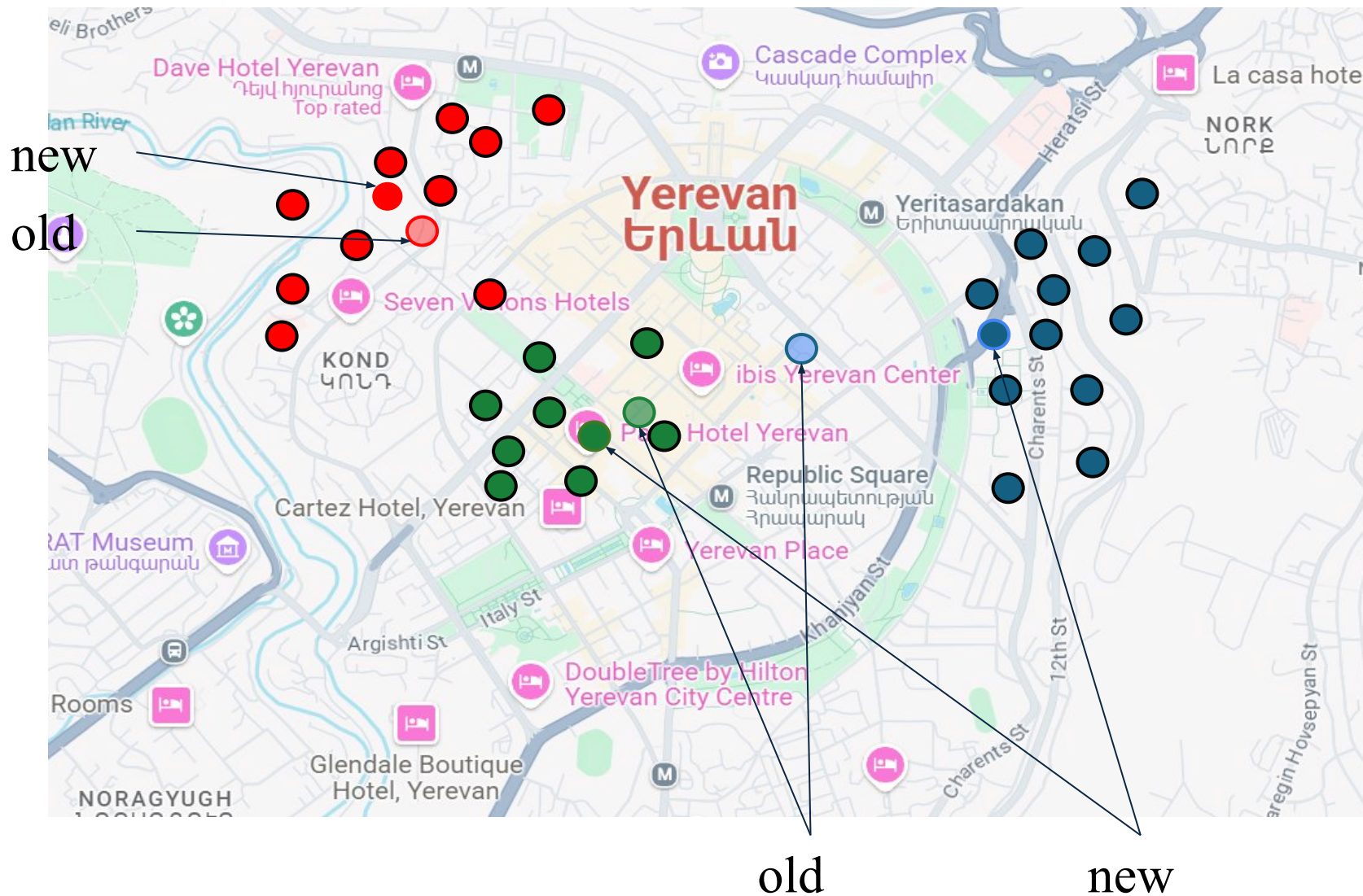


Repeat the process for all data points.



Repeat for all points. Each point now belongs to a cluster i.e. it is labeled as 1,2,3 or by their corresponding colors.

Step 5: Compute the mean of each cluster.



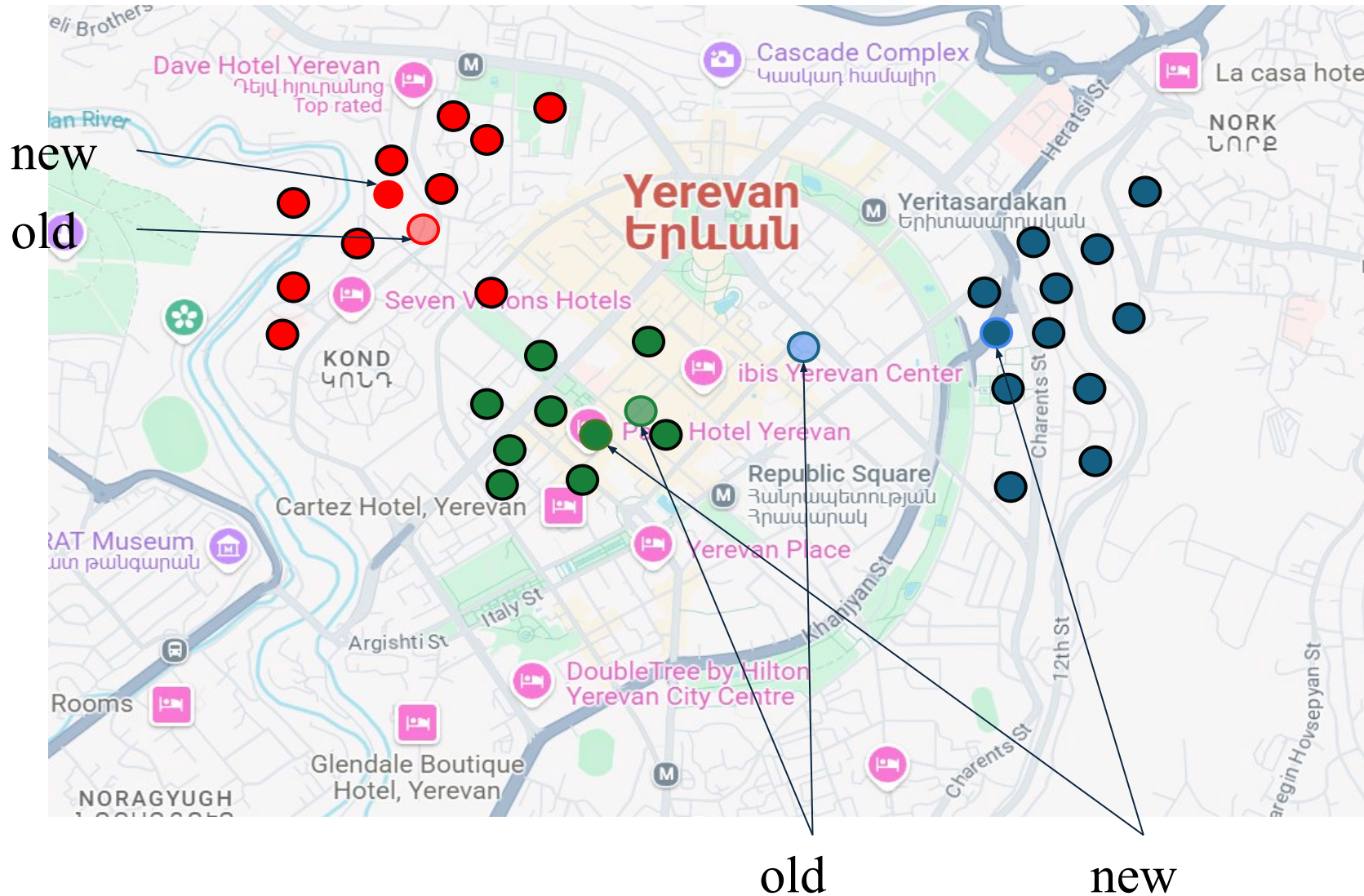
For each cluster j , compute the mean (average) of all the data points assigned to it. This “mean” becomes the new cluster center.

$$\mu_x^j = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\mu_y^j = \frac{1}{n} \sum_{i=1}^n y_i$$

New center (μ_x^j, μ_y^j)

Step 6: Compute the variance of each cluster.

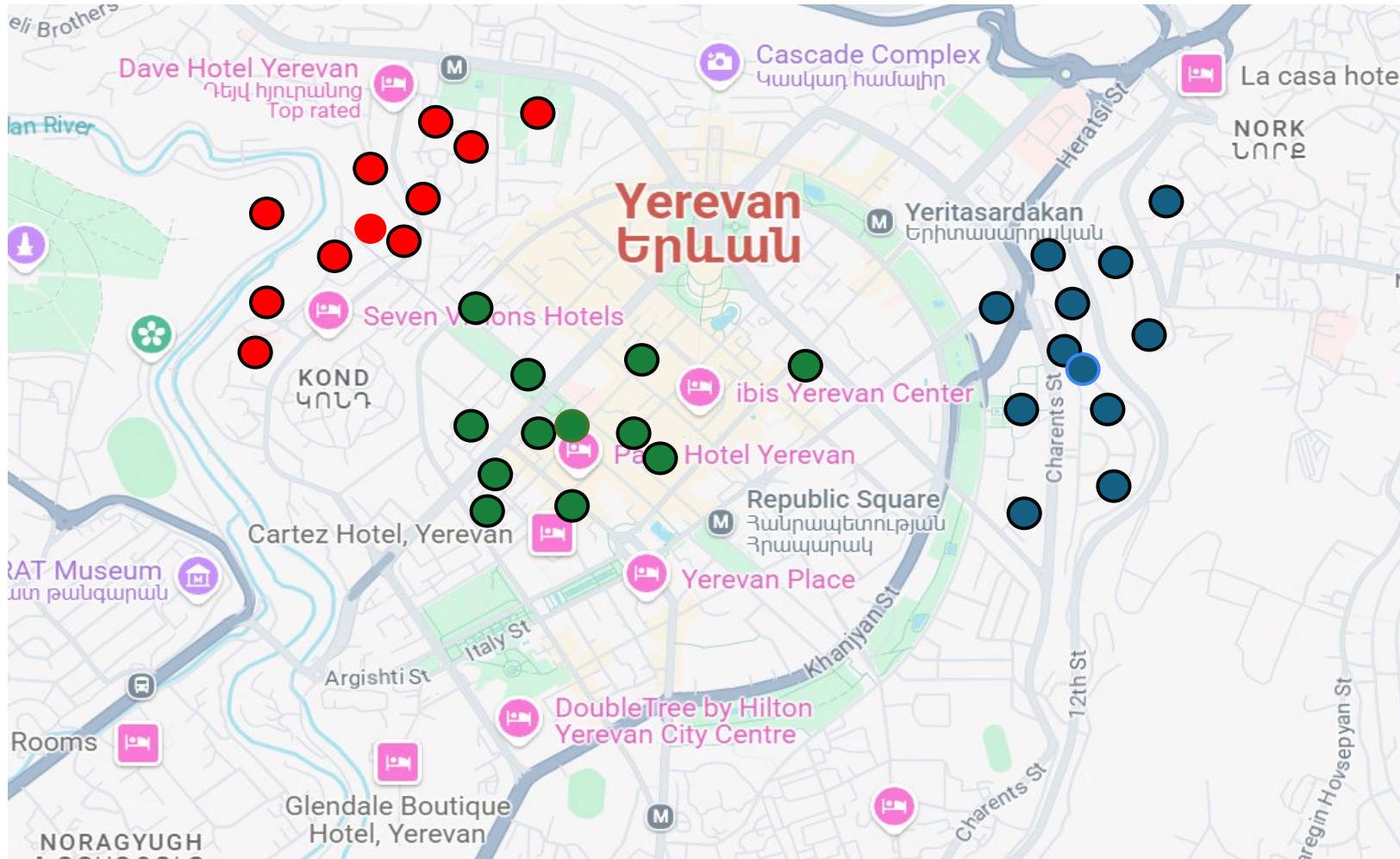


Calculate the variation (e.g., within-cluster variance) to assess how compact each cluster j is.

$$\sigma_j^2 = \frac{1}{n_k} \sum_{i \in C_j} |x_i - \mu_j|^2$$

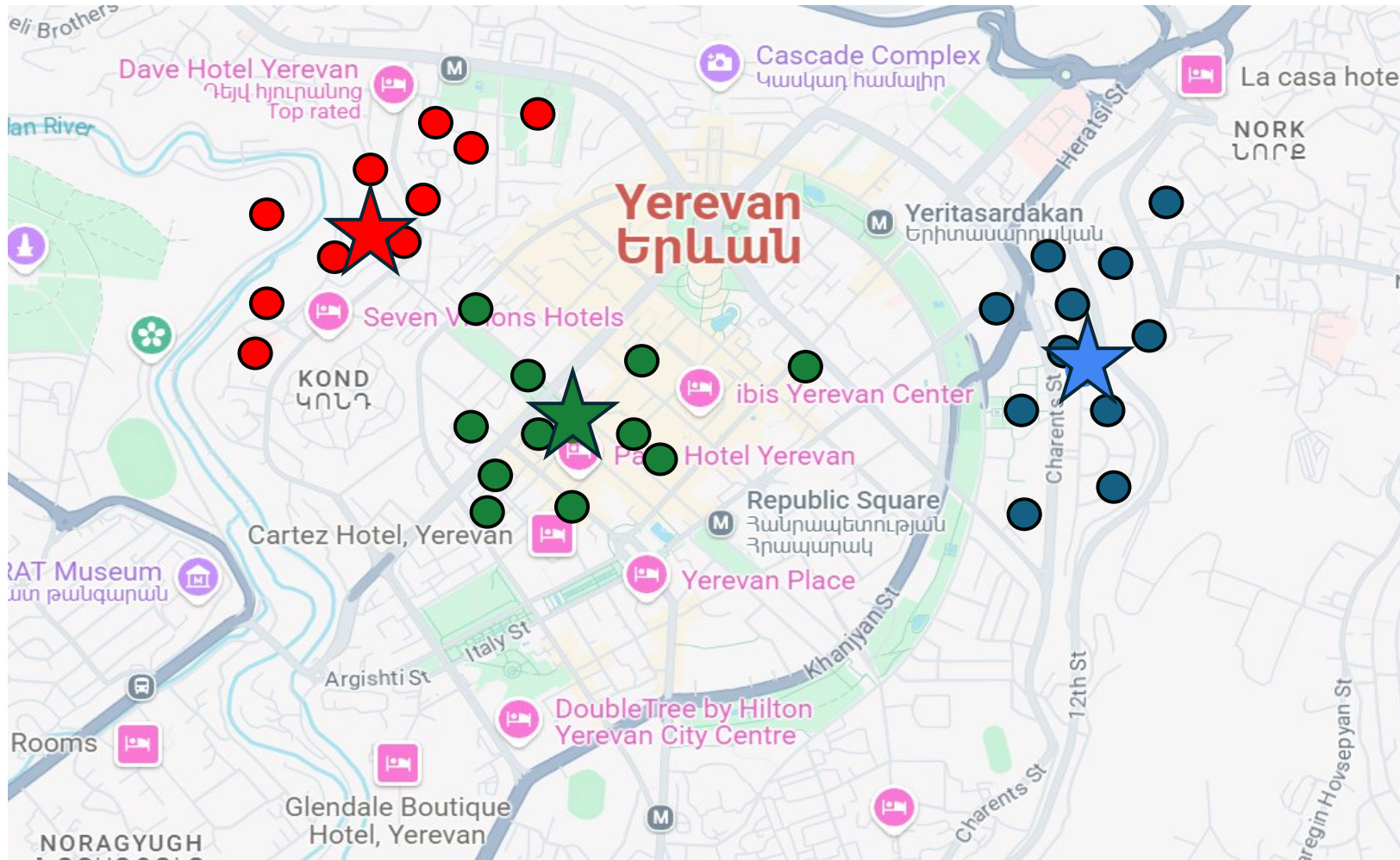
for all point $\mathbf{x}_i$ that are currently in cluster $\mathbf{c}_j$.

Step 7: Check for convergence.



If the cluster assignments no longer change or the centers remain the same, stop the algorithm. Otherwise, return to Step 3 with the updated centers.

Final Pizza Delivery Centers!



To avoid poor initialization, repeat the process multiple times with different starting points and select the result with the lowest overall variance.

k-Means Clustering

Easily explained
& calculated



But how can we choose the number of clusters k ?

Determining the optimal number of clusters is a common challenge in clustering tasks. Several techniques can help estimate a suitable value, including the *Elbow Method*, which runs the k-means algorithm for different values of k and identifies a point where increasing k no longer significantly improves the clustering.

- This point represents a balanced choice for the number of clusters.

Elbow Method Algorithm

Choose a range for **k**

Decide on a range of cluster numbers to try,
e.g., **k = 1 to k = 10**.

Apply K-Means for each **k**

Run the k-means algorithm.
For each run, record the inertia
(also called within-cluster sum
of squares or WCSS):

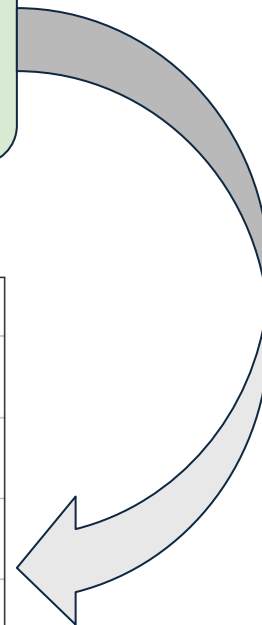
$$Inertia = \sum_{j=1 \text{ to } k} \sum_{i \in C_k} |x_i - \mu_j|^2$$

Elbow Method Algorithm

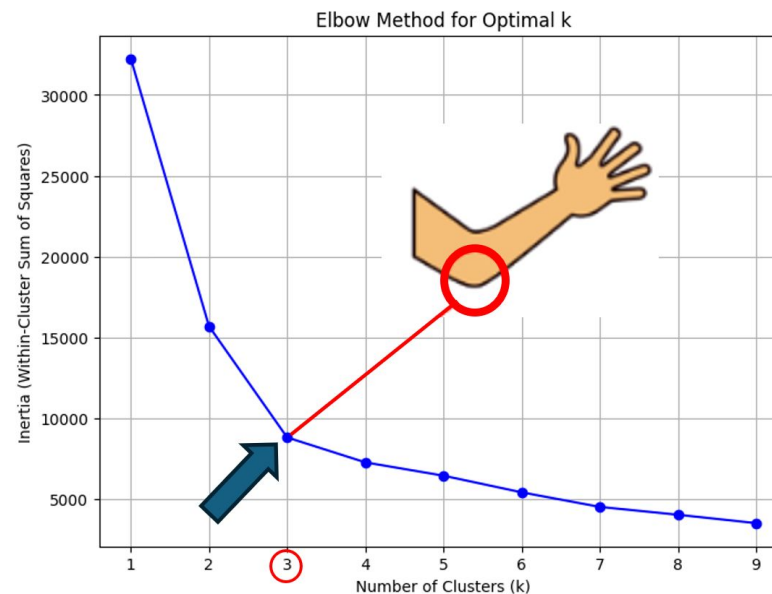
Plot K vs Inertia



Identify the “elbow point”



- Create a plot where the x-axis is the number of clusters k , and the y-axis is the corresponding inertia.
- The inertia usually decreases as k increases (more clusters = less dispersion within each).



Select optimal k

Session 3: The K-Means Algorithm

Հեղինակային իրավունքը պատկանում է «Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամ»-ին և ենթակա չէ ազատ տարածման:

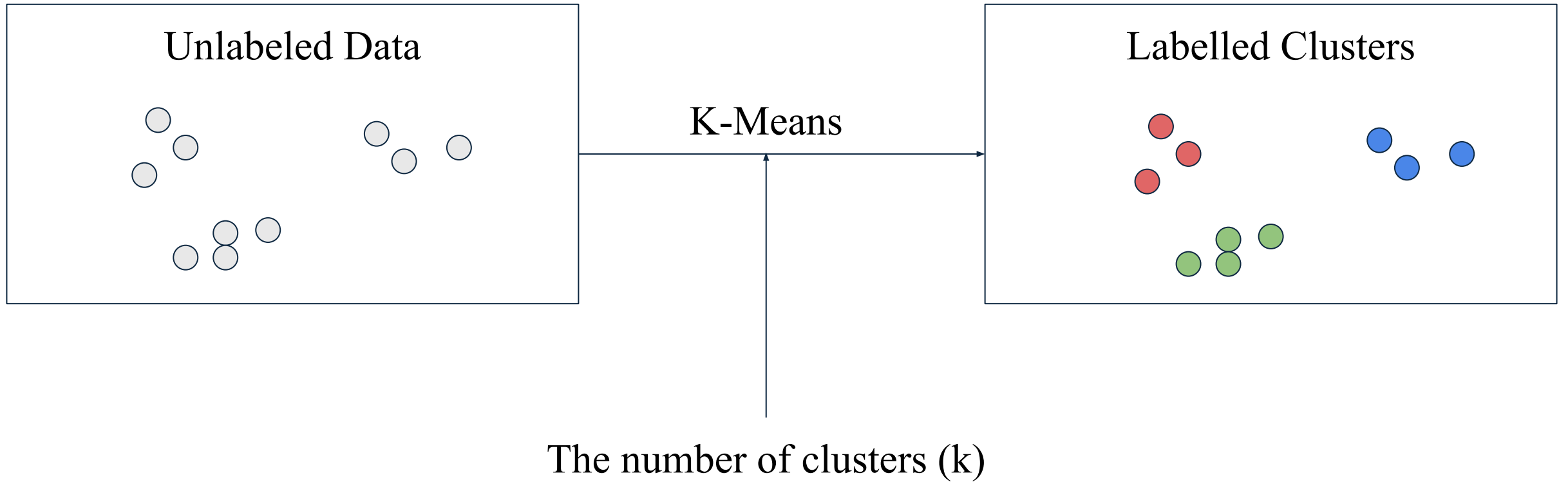


The Objective of the Session

- Apply K-Means on synthetic data.



Reminder



Key Attributes

labels_ - An array of integers (shape: same as number of samples in X) indicating the cluster each sample belongs to. Same as what you get from *fit_predict(X)*.

cluster_centers_ - Coordinates of the cluster centers.

inertia_ - Sum of squared distances of samples to their closest cluster center.

n_iter_ - Number of iterations run by the algorithm until convergence.

source: [link](#).

Key Attributes

fit(X, y=None, sample_weight=None) - Fit the model on data X.

predict(X) - Predict the closest cluster each sample in X belongs to.

fit_predict(X, y=None, sample_weight=None) - Equivalent to calling fit(X) followed by predict(X).

transform(X) - Returns the distance of each sample in X to each cluster center.

fit_transform(X, y=None, sample_weight=None) - Equivalent to fit(X).transform(X), but more efficiently implemented.

source: [link](#).

Coding

```
# Set the number of clusters to 3
k = 3

# Create a KMeans clustering model with 3 clusters and a fixed random seed for reproducibility
kmeans = KMeans(n_clusters=k, random_state=42)

# Fit the KMeans model to the data 'X' and assign each data point to a cluster
# The result is an array of cluster labels (0, 1, or 2) corresponding to each data point in X
labels = kmeans.fit_predict(X)

# Get the coordinates of the cluster centers (centroids)
# Each row in 'centroids' represents the center of one cluster in the same feature space as X
centroids = kmeans.cluster_centers_
```

Coding

```
# Initialize an empty list to store inertia values for each k
inertias = []

# Define the range of cluster numbers to try (from 1 to 9)
k_range = range(1, 10)

# Loop over the range of k values
for k in k_range:
    # Create a KMeans model with the current number of clusters
    kmeans = KMeans(n_clusters=k, random_state=42)

    # Fit the model to the data
    kmeans.fit(X)

    # Append the inertia (sum of squared distances to the nearest cluster center) to the list
    inertias.append(kmeans.inertia_)
```


Colab Session

Access the COLAB [link](#) and work through the tasks.

Session 4: Colab Session: The K-Means vs DBSCAN Algorithm 2

Հեղինակային իրավունքը պատկանում է «Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամ»-ին և ենթակա չէ ազատ տարածման:



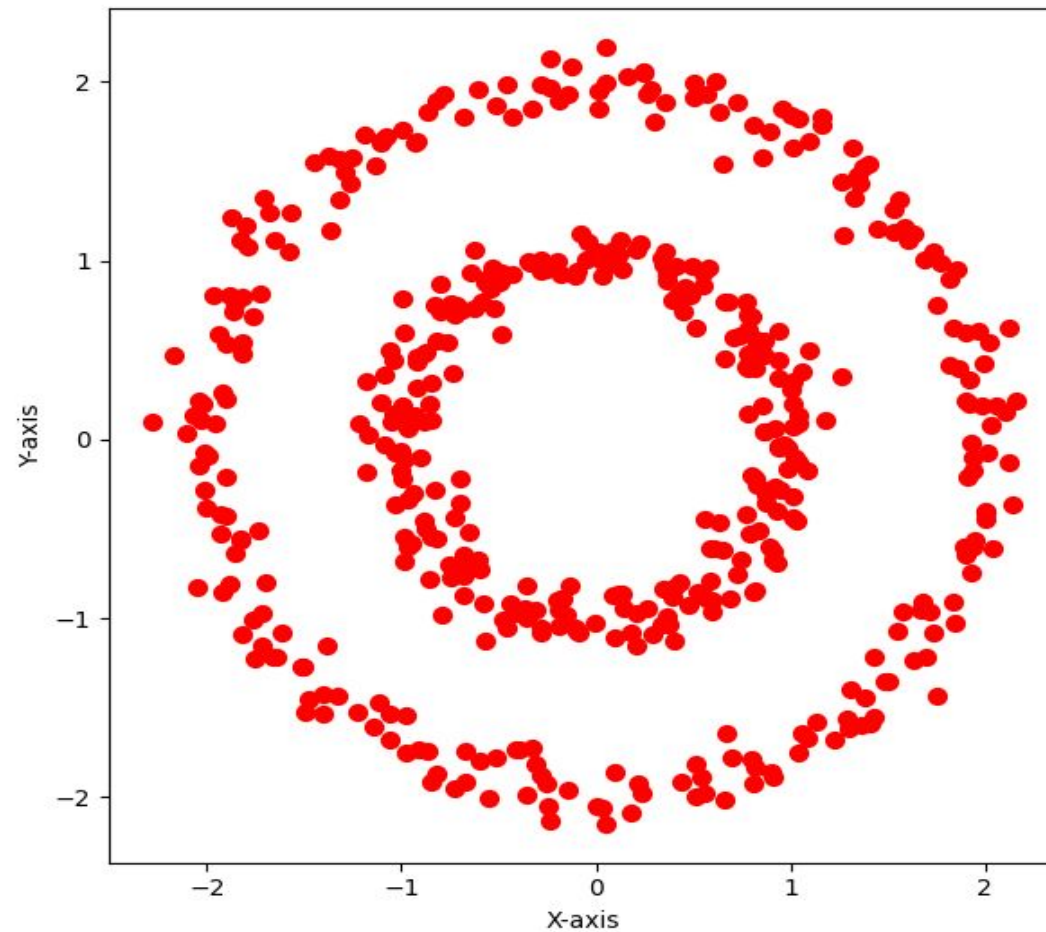
The Objective of the Session

- Apply k-means on irregular data.
- Introduce DBSCAN.
- Compare DBSCAN and k-means.



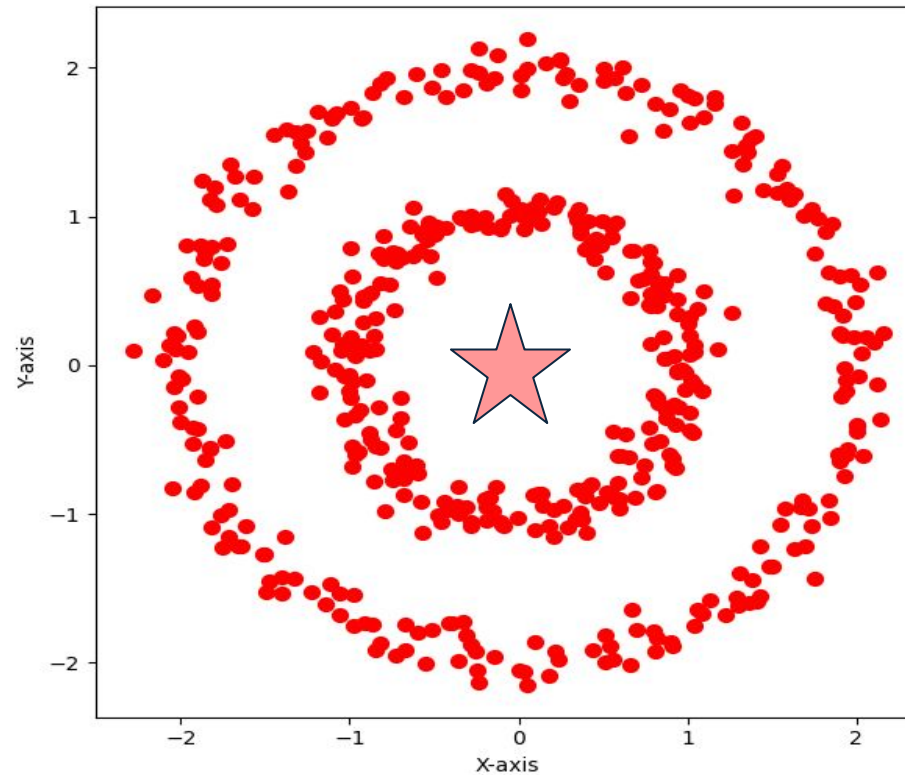
K-Means: Is it always the right tool?

What can be the result of applying K-Means on circular data like in this task?



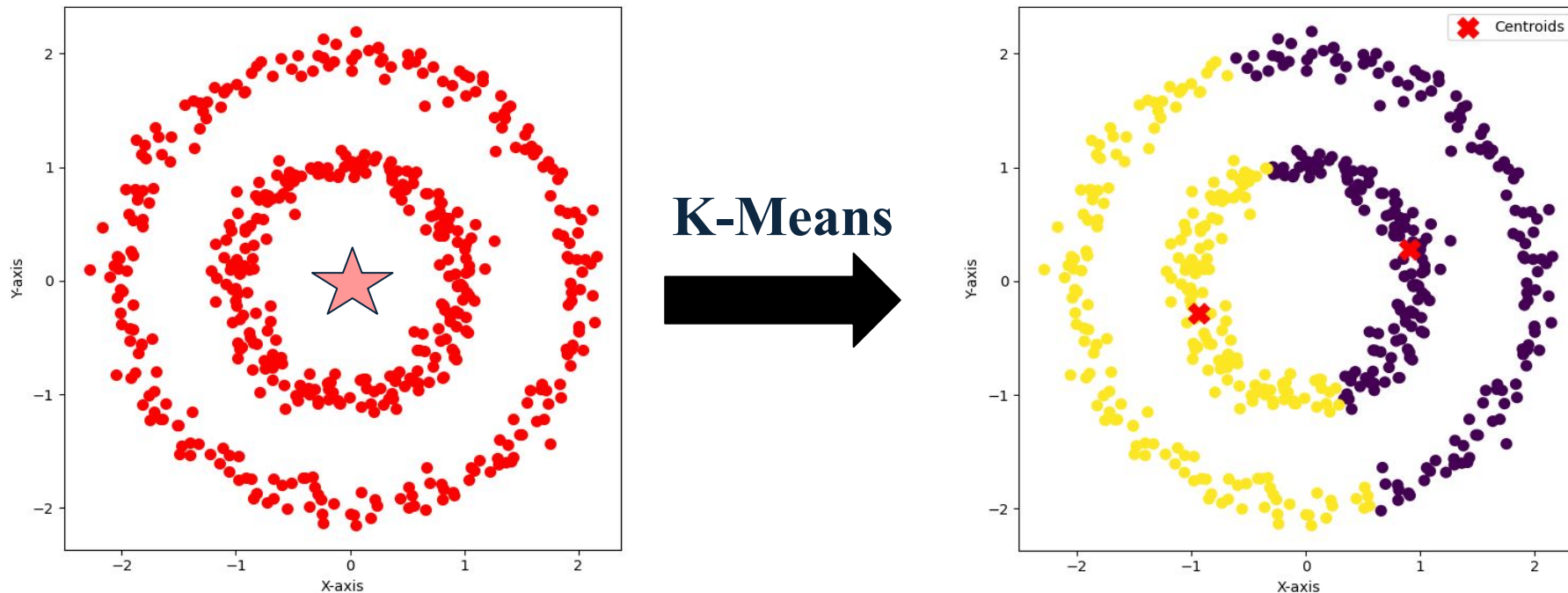
K-Means: Is it always the right tool?

When the data is arranged in a circular or ring-like structure, the K-Means fails. This is because K-Means relies on calculating the centroid, or average position, of the data points in each cluster. In circular distributions, the true "centers" of the clusters may overlap or lie



K-Means: Is it always the right tool?

Since the algorithm assigns points to the nearest centroid based on Euclidean distance, it ends up misclassifying many points, producing clusters that do not accurately reflect the underlying pattern.

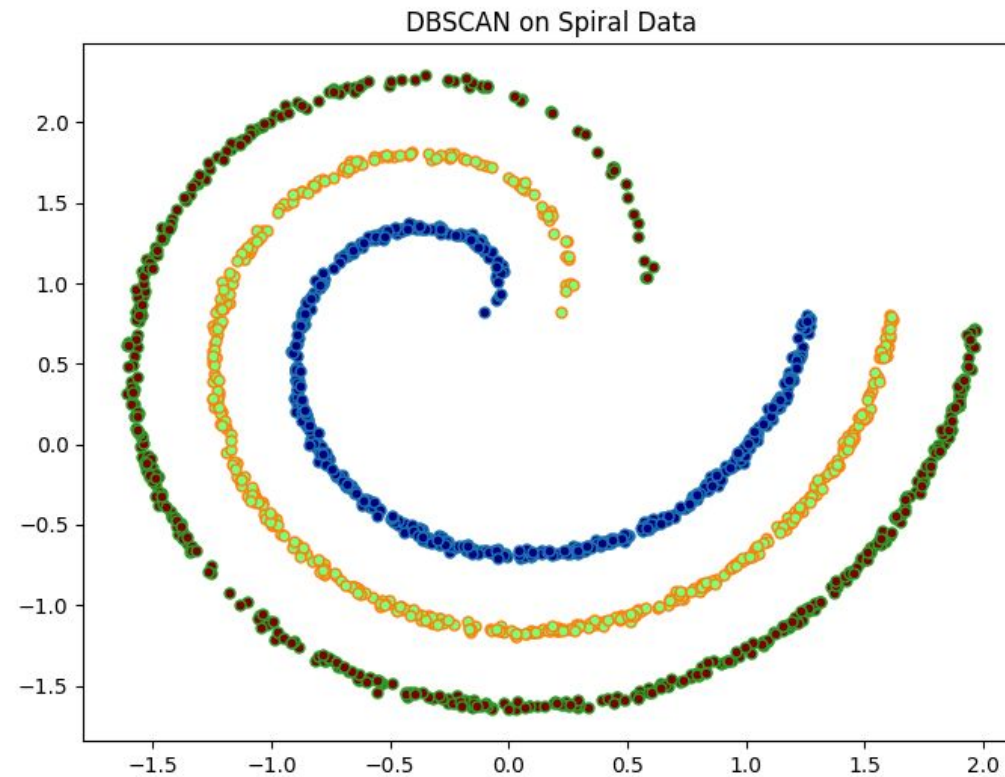


DBSCAN

In such situations, DBSCAN (Density-Based Spatial Clustering of Applications with Noise) can serve as a more robust alternative to K-Means. Unlike K-Means, which assumes clusters are spherical and separates them based on distance to centroids, ***DBSCAN groups data based on density***. It defines clusters as areas of high point density separated by regions of low density. This makes it well-suited for identifying clusters of arbitrary shapes, such as rings or spirals.

DBSCAN

DBSCAN does not require the number of clusters to be specified in advance and is also effective at identifying and excluding noise or outliers.



DBSCAN: Algorithm

It *starts with one point* and checks how many other points are within a certain distance (called epsilon).

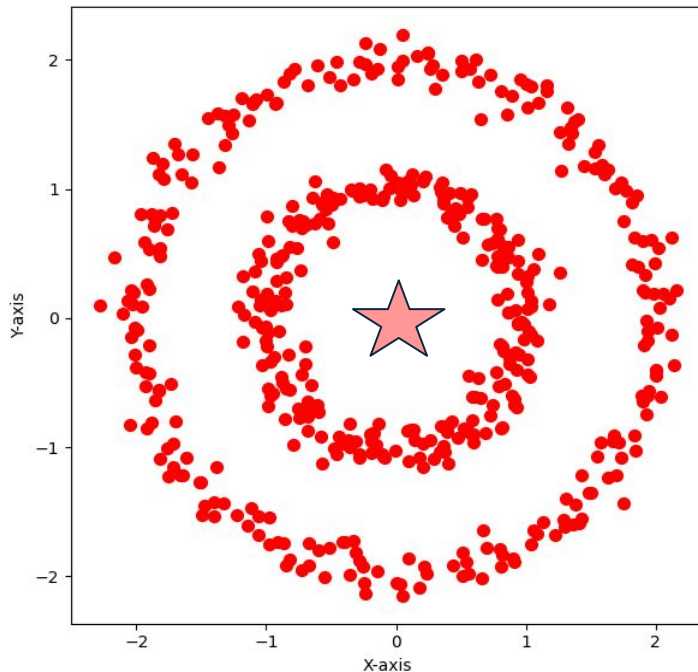
If there are enough *nearby points* (at least a minimum number called minPts), it's considered part of a dense region.

DBSCAN then *grows the cluster* by checking *neighbors of neighbors* — slowly expanding outward to include all reachable dense areas.

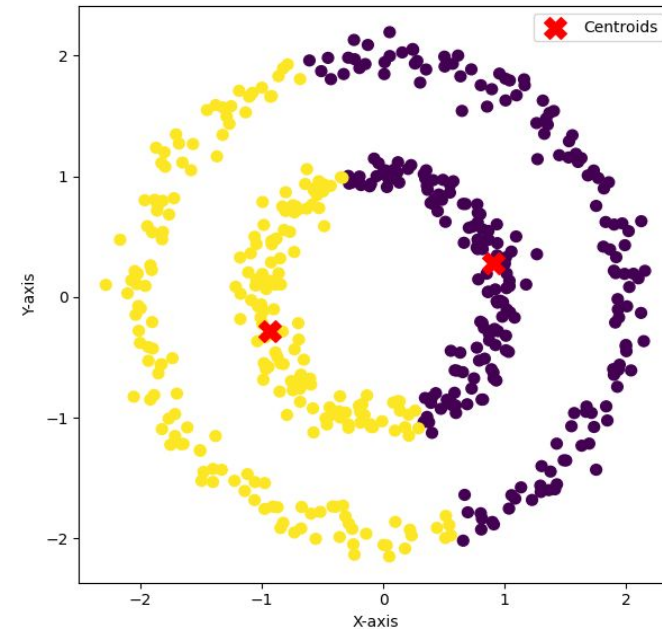
If a point doesn't have enough nearby neighbors and can't be reached from a dense region, it's considered noise or an outlier.

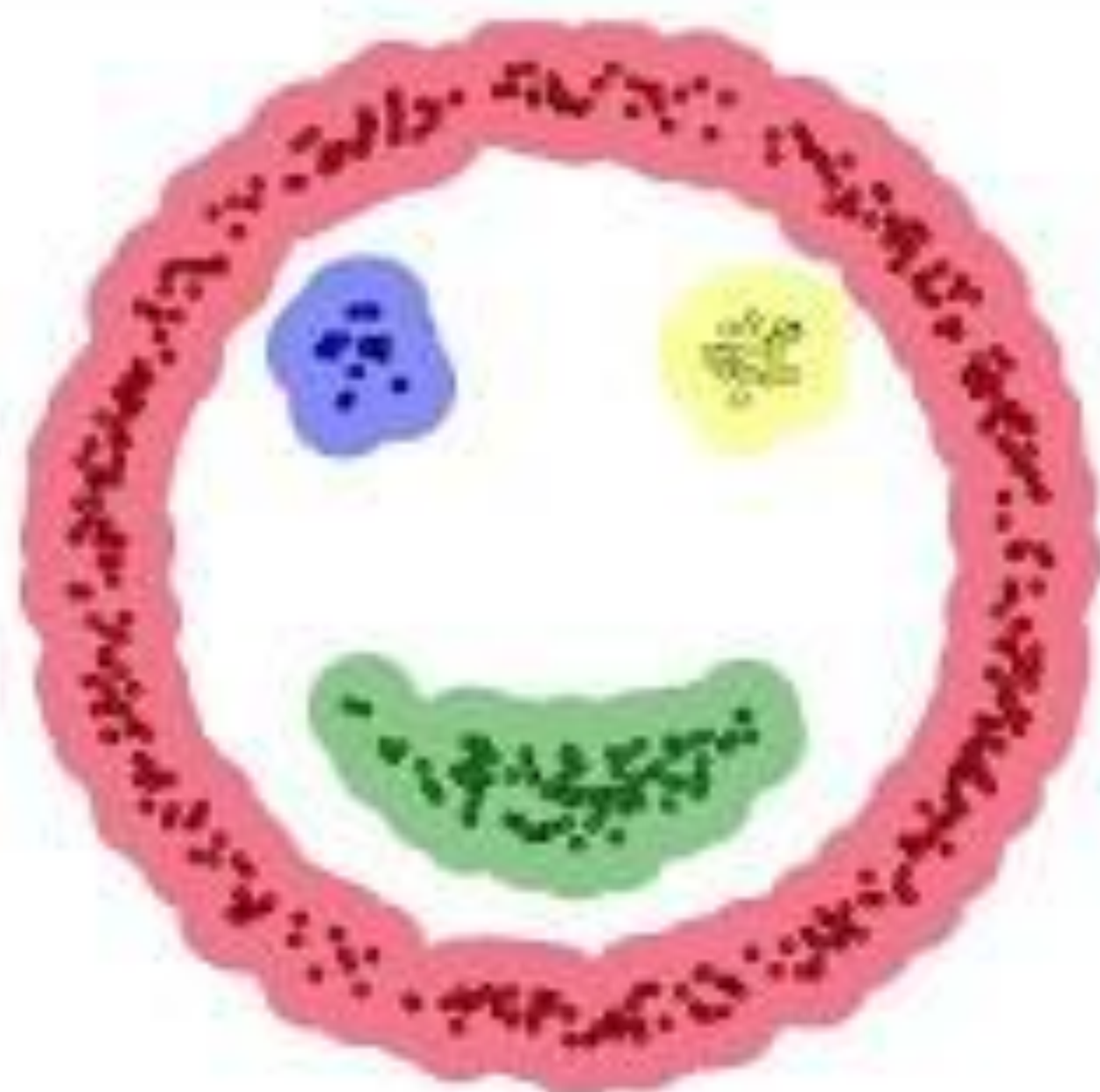
DBSCAN

DBSCAN grows clusters by exploring dense neighborhoods, making it ideal for detecting irregular or non-convex shapes and ignoring outliers. Unlike k-means, it doesn't assume spherical clusters or require a preset number of clusters. This makes DBSCAN especially effective for complex structures, like circular or spiral data, where traditional methods often fail.



K-Means





epsilon = 1.10
minPoints = 4

Colab Session

Access the COLAB [link](#) and work through the tasks.

Session 5

- Summary